Need of Collection:

Int x=10;int y=20;int z=30;

To store huge no of values we use array.

Student[] s=new Student[10000];

Limitations of Array:

1.Fixed Size 2. Homogeneous Elements 3. No Underlying data structure so readymade method support is not there.

By Above Student array we can represent only student type of object.

s[0]=new Student();

S[1]=new Customer() // RE: incompatible type found :Customer required :Student

Object[] a=new Object[10000];

a[0]=new Student();

A[1]=new Customer();

Collections: to overcome the problems of array we should go for collections.

1. Collections are growable in nature.
2. Homogeneous and heterogeneous Objects
3. Based on Some underlying Standard data Structure so readymade methods are available.

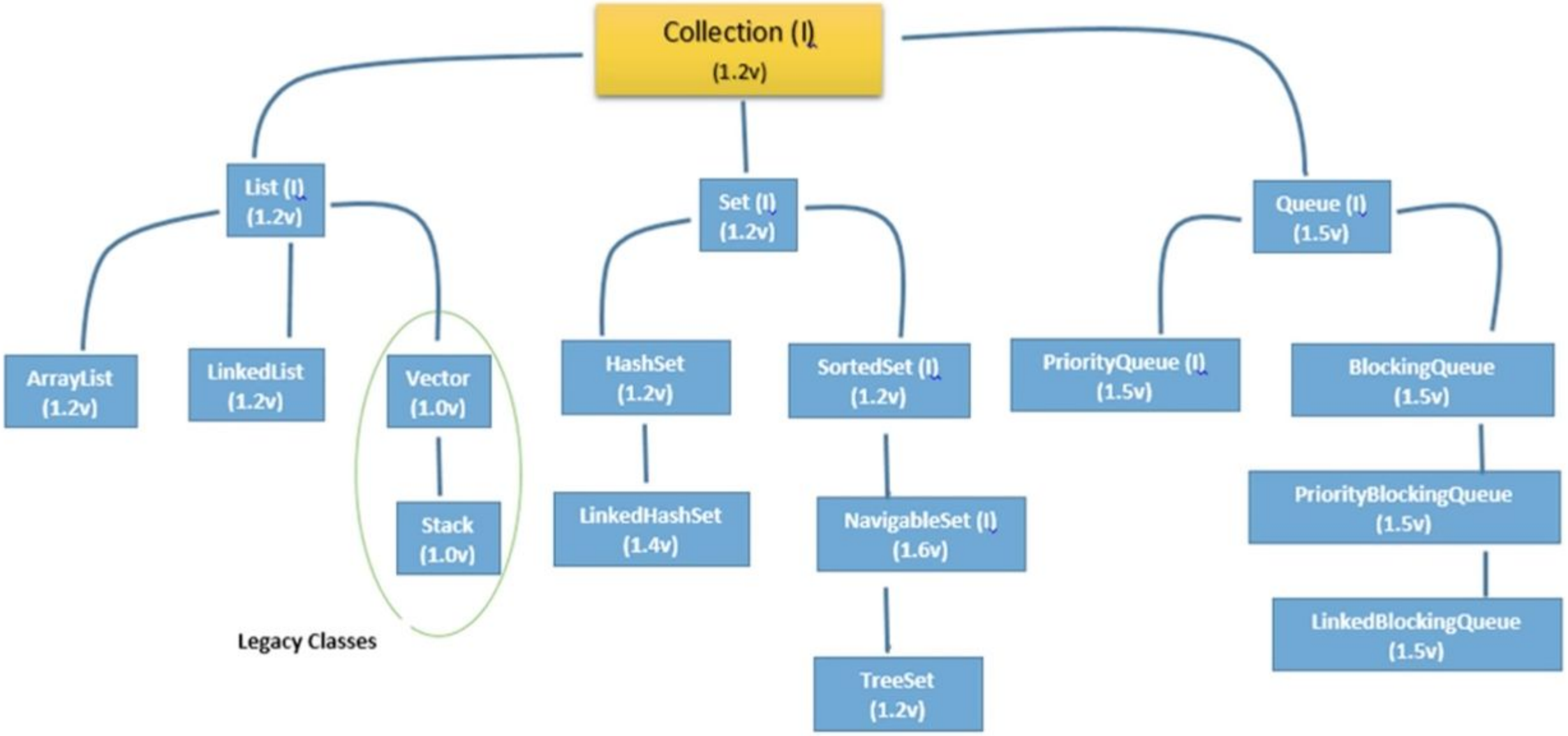
Definition: Collection is a group of individual objects as a single entity. If you want to represent a group of individual objects as a single entity then we should go for collection.

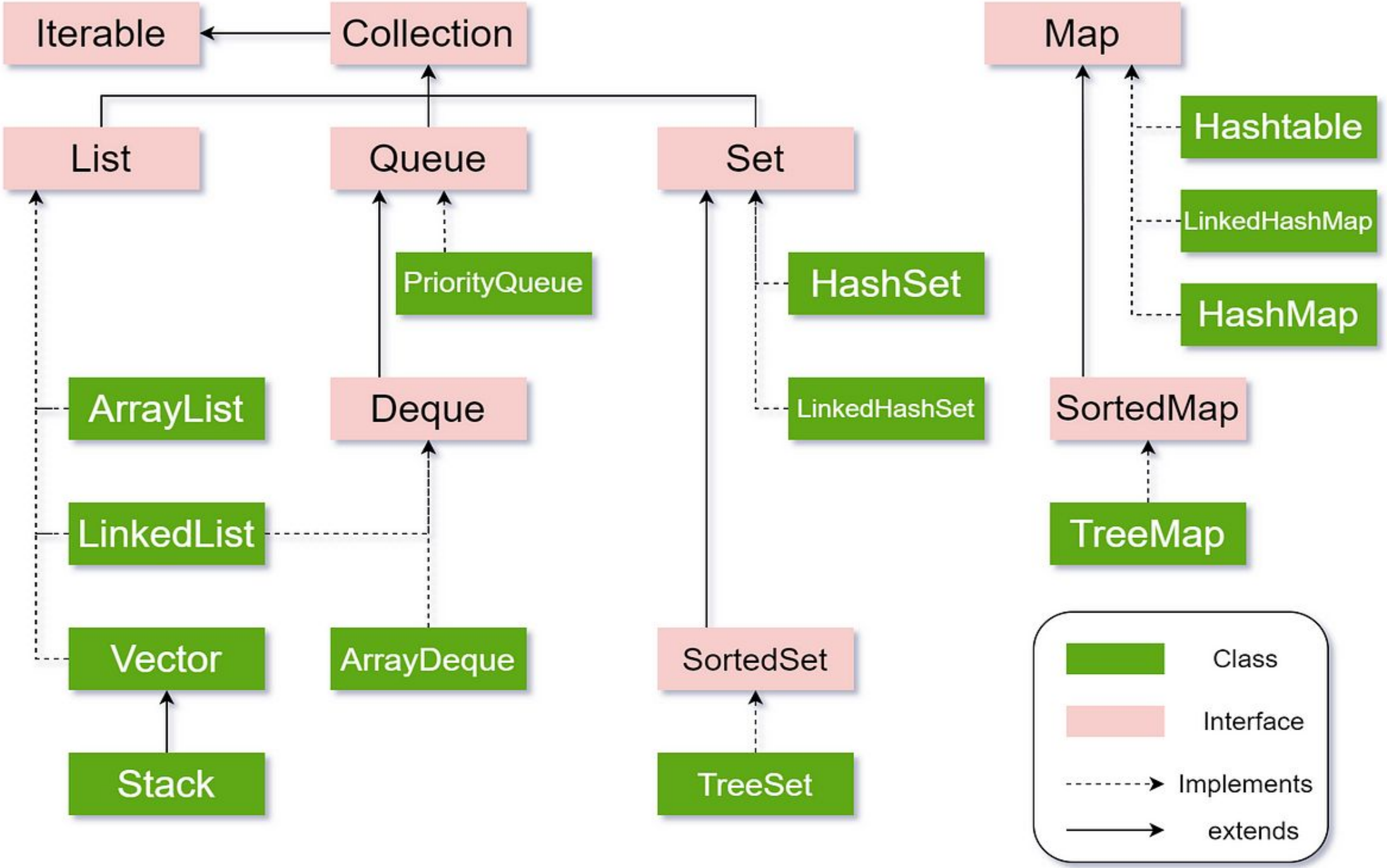
Collection Framework : It define several classes and interfaces which can be used to represent a group of individual objects as a single entity.

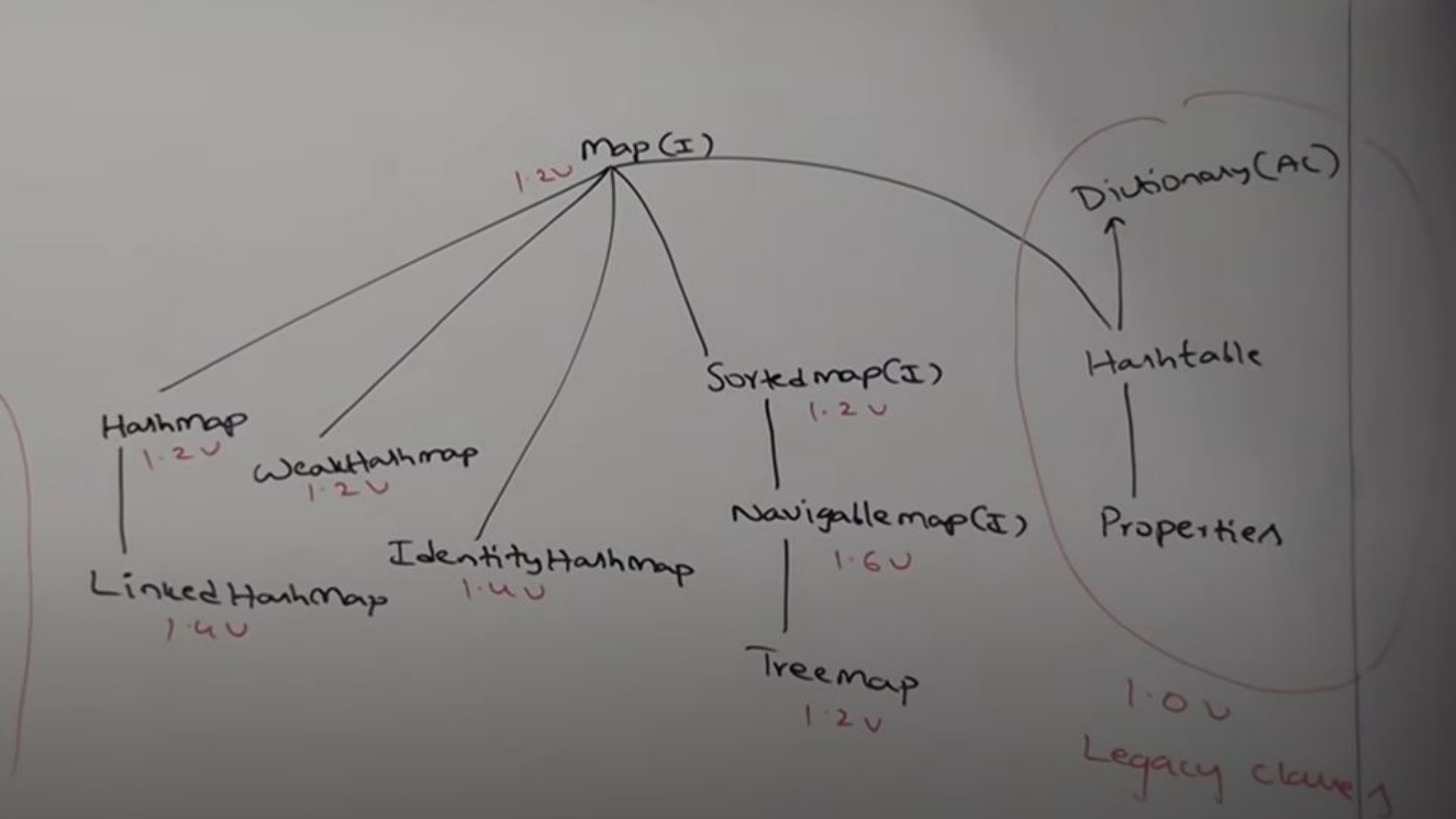
In collection framework there are total 9 Key interfaces.

1. Collection(I):This interface defines the most common methods which are applicable for any collection objects. Collection interface is considered as root interface in collection framework. Example: add(),remove(),contains(),isEmpty()

Difference between Collection and Collections: Collection is interface but Collections is a utility class which defined various utility methods for collection objects. For Example:


Collection (I)
(1.2v)
List (I)
(1.2v)
Set (I)
(1.2v)
Queue (I)
(1.5v)
ArrayList
(1.2v)
LinkedList
(1.2v)
Vector
(1.0v)
Stack
(1.0v)
Legacy Classes
HashSet
(1.2v)
LinkedHashSet
(1.4v)
SortedSet (I)
(1.2v)
NavigableSet (I)
(1.6v)
TreeSet
(1.2v)
PriorityQueue (I)
(1.5v)
BlockingQueue
(1.5v)
PriorityBlockingQueue
(1.5v)
LinkedBlockingQueue
(1.5v)


Iterable
Collection
Map
List
Queue
Set
Hashtable
LinkedHashMap
HashMap
PriorityQueue
HashSet
ArrayList
Deque
LinkedHashSet
SortedMap
LinkedList
TreeMap
Vector
ArrayDeque
SortedSet
Stack
TreeSet
Class
Interface
Implements
extends


Map(I)
1.2v
Hashmap
1.2v
WeakHashmap
1.2v
IdentityHashmap
1.4v
LinkedHashMap
1.4v
SortedMap(I)
1.2v
NavigableMap(I)
1.6v
Treemap
1.2v
Dictionary(AC)
Hashtable
Properties
1.0v
Legacy class

Collection(I):When we want to represent a group of individual object as a single

Entity then we should go for collection.

List(I): If in the group of individual objects duplicates are allowed and insertion order must be preseved. Then we should go for list interface.

Implementation classes for List interface is ArrayList, LinkedList,Vector and Stack.

Set(I): If in the group of individual objects duplicates are not allowed. Insertion order is not preserved then we should go for Set interface.

Implementation classes for Set(I) is HashSet, LinkedHashSet

SortedSet(I):If we do not want duplicates but sorting order is required.

NavigableSet(I): Navigation support (Which is previous or next element) if we want yhen we should go for NavigableSet interface.

Implementation class for NAvigableSet is TreeSet

Queue(I): Before processing if you want to represent group of individual object as a single entity then you should go for queue. For example if you want to store all mobile numbers before sending SMS.

Implementation classes for Queue is:

Priority Queue

Blocking Queue

Priority Blocking Queue

Linked Blocking Queue

.

.

If you want to represent a group of individual objects as key-value pair then we should go for Map interface.

Sorting:

Comparable(I) is used for default natural sorting order.

Comparator(I) is used for Customized sorting order.

Cursors: are used to access collection objects one by one.

Enumeration(I)

Iterator(I)

ListIterator(I)

Utility Class:

Collections

Arrays

Cursors:1. Enumeration:

public Enumeration elements();

Enumeration e=v.elements();

Methods:

public Boolean hasMoreElements()

public Object nextElement();

Example:

```
Vector v=new Vector();
for(int i=0;i<=10;i++){
v.addElement(i);}
Sop(v);
Enumeration e=v.elements();
While(e.hasMoreElemts()){
```

```
Integer I=(Integer)e.nextElement();
If(I%2==0)
}
Sop(I);
```

2. Iterator(I):

```
public Iterator iterator();
Iterator itr=c.iterator();
```

Methods:

```
public boolean hasNext(0
public Object next()
public void remove()
```

Example:

```
AL l=new AL();
for(int i=0;i<=10;i++){
l.add(i);}
Sop(v);
Iterator itr=l.iterator();
```

```
While(itr.hasNext()){
Integer I-(Integer)itr.next();
if(I%2==0)
Sop(I);
else
itr.remove();
}
```

2. List(I)(1.2): List interface is Child interface of Collection. If in your groups of individual objects duplicates are allowed , insertion ordered must be preserved then we should go for List Interface. Index are used to differentiate duplicates. Insersetion order are preserve using index.

Various specific Method:

add(int index,Object o), addAll(int index, Collection c), remove(int index), indexOf("A"), lastIndexOf("A"), get(int index), set(int index,Object o)

ListIterator listIterator(),

Implementation classes for List Interface:

ArrayList(1.2)

LinkedList(1.2)

Vector(1.0)----▫Stack(1.0) Legacy Classes

2. ArrayList(C)(1.2): It is Implementation class of List Interface. Resizable array or Growable array is underlying data structure on which ArrayList is based.

- Duplicated allowed. Insertion order preserved. Heterogeneous Objects are allowed.
- Null insertion is possible.
- RandomAccess interface already implemented. No method of ArrayList are synchronized.
- ArrayList is not Thread Safe. Every Collection class already implements Serializable and cloneable interface.

Note: Except TreeSet and TreeMap everywhere heterogeneous objects are allowed.

Constructors:

1. ArrayList l=new ArrayList();//Default inistial capacity is 10.

Note: New capacity=(current capcity*3/2)+1

2. ArrayList l=new ArrayList(int initial capacity)

3. ArrayList l=new ArrayList(Collection c)

```
import java.util.*;

class ArrayListDemo{

public static void main(String[] args){

ArrayList l=new ArrayList();

l.add(“A”);l.add(10);l.add(“A”);

l.add(null);

System.out.println(l);

l.remove(2);

 System.out.println(l);

l.add(2,”M”);l.add(“N”);

System.out.println(l);}}
```

Java **ArrayList** class uses a *dynamic array* for storing the elements. It is like an array, but there is *no size limit*. We can add or remove elements anytime. So, it is much more flexible than the traditional array. It is found in the *java.util* package. It is like the Vector in C++.

The ArrayList in Java can have the duplicate elements also. It implements the List interface so we can use all the methods of the List interface here. The ArrayList maintains the insertion order internally.

It inherits the AbstractList class and implements List interface.

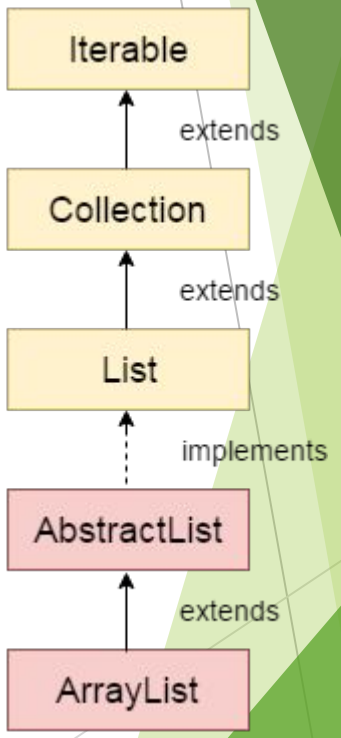
The important points about the Java ArrayList class are:

- Java ArrayList class can contain duplicate elements.
- Java ArrayList class maintains insertion order.
- Java ArrayList class is non synchronized.
- Java ArrayList allows random access because the array works on an index basis.
- In ArrayList, manipulation is a little bit slower than the LinkedList in Java because a lot of shifting needs to occur if any element is removed from the array list.
- We can not create an array list of the primitive types, such as int, float, char, etc. It is required to use the required wrapper class in such cases. For example:

1. ArrayList<int> al = ArrayList<int>();

Note: When retrieval is frequent Operation ArrayList is best choice because it implemented RandomAccess Interface.

It is worst choice if frequent insertion and deletion in middle is required.



Java Non-generic Vs. Generic Collection

Java collection framework was non-generic before JDK 1.5. Since 1.5, it is generic.

Java new generic collection allows you to have only one type of object in a collection. Now it is type-safe, so typecasting is not required at runtime.

Let's see the old non-generic example of creating a Java collection.

```
1. ArrayList list=new ArrayList();//creating old non-generic arraylist
```

Let's see the new generic example of creating java collection.

```
1. ArrayList<String> list=new ArrayList<String>();//creating new generic arraylist
```

In a generic collection, we specify the type in angular braces. Now ArrayList is forced to have the only specified type of object in it. If you try to add another type of object, it gives a *compile-time error*.

```
import java.util.*;
 public class ArrayListExample1{
 public static void main(String args[]){
  ArrayList<String> list=new ArrayList<String>();//Creating arraylist
     list.add("Mango");//Adding object in arraylist
     list.add("Apple");
     list.add("Banana");
     list.add("Grapes");
     //Printing the arraylist object
     System.out.println(list);
 }
}
```

Iterating ArrayList using Iterator

```
import java.util.*;
public class ArrayListExample2{
 public static void main(String args[]){
  ArrayList<String> list=new ArrayList<String>();//Creating arraylist
  list.add("Mango");//Adding object in arraylist
  list.add("Apple");
  list.add("Banana");
  list.add("Grapes");
  //Traversing list through Iterator
  Iterator itr=list.iterator();//getting the Iterator
  while(itr.hasNext()){//check if iterator has the elements
   System.out.println(itr.next());//printing the element and move to next
  }
 }
}
```

Iterating ArrayList using For-each loop

```
import java.util.*;
public class ArrayListExample3{
 public static void main(String args[]){
  ArrayList<String> list=new ArrayList<String>();//Creating arraylist
  list.add("Mango");//Adding object in arraylist
  list.add("Apple");
  list.add("Banana");
  list.add("Grapes");
  //Traversing list through for-each loop
  for(String fruit:list)
    System.out.println(fruit);

 }
}
```

Get and Set ArrayList

The *get() method* returns the element at the specified index, whereas the *set() method* changes the element.

```
import java.util.*;
public class ArrayListExample4{
 public static void main(String args[]){
  ArrayList<String> al=new ArrayList<String>();
  al.add("Mango");
  al.add("Apple");
  al.add("Banana");
  al.add("Grapes");
  //accessing the element
  System.out.println("Returning element: "+al.get(1));//it will return the 2nd element, because index starts fr
om 0
  //changing the element
  al.set(1,"Dates");
  //Traversing list
  for(String fruit:al)
    System.out.println(fruit);

 }
}
```

How to Sort ArrayList

The *java.util* package provides a utility class **Collections**, which has the static method sort(). Using the **Collections.sort()** method, we can easily sort the ArrayList.

```
import java.util.*;
class SortArrayList{
 public static void main(String args[]){
  //Creating a list of fruits
  List<String> list1=new ArrayList<String>();
  list1.add("Mango");    list1.add("Apple");
  list1.add("Banana");    list1.add("Grapes");
    Collections.sort(list1);  //Sorting the list
   //Traversing list through the for-each loop
  for(String fruit:list1)
    System.out.println(fruit);
       System.out.println("Sorting numbers...");
    List<Integer> list2=new ArrayList<Integer>();  //Creating a list of numbers
  list2.add(21);    list2.add(11);
  list2.add(51);
  list2.add(1);
  Collections.sort(list2);   //Sorting the list
   //Traversing list through the for-each loop
  for(Integer number:list2)
    System.out.println(number);  }}
```

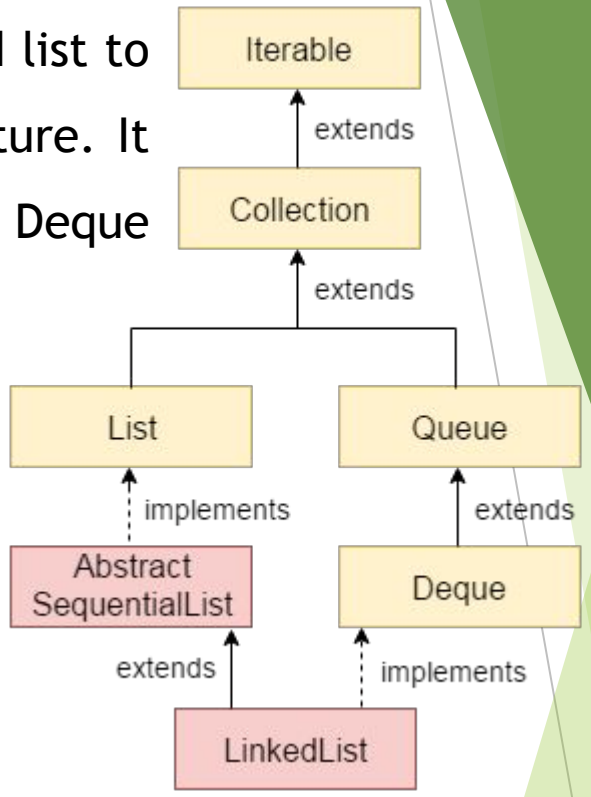
Ways to iterate the elements of the collection in Java

There are various ways to traverse the collection elements:

1. By Iterator interface.
2. By for-each loop.
3. By ListIterator interface.
4. By for loop.
5. By forEach() method.
6. By forEachRemaining() method.

```
import java.util.*;
class ArrayList4{
 public static void main(String args[]){
   ArrayList<String> list=new ArrayList<String>();//Creating arraylist
        list.add("Ravi");//Adding object in arraylist
        list.add("Vijay");          list.add("Ravi");          list.add("Ajay");
        System.out.println("Traversing list through List Iterator:");
        //Here, element iterates in reverse order
          ListIterator<String> list1=list.listIterator(list.size());
          while(list1.hasPrevious())  {
              String str=list1.previous();
              System.out.println(str);       }
      System.out.println("Traversing list through for loop:");
        for(int i=0;i<list.size();i++)          {
         System.out.println(list.get(i));             }
      System.out.println("Traversing list through forEach() method:");
      //The forEach() method is a new feature, introduced in Java 8.
         list.forEach(a->{ //Here, we are using lambda expression
            System.out.println(a);        });
         System.out.println("Traversing list through forEachRemaining() method:");
          Iterator<String> itr=list.iterator();
          itr.forEachRemaining(a-> //Here, we are using lambda expression
          {
         System.out.println(a);             });  }  }
```

LinkedList:When our frequent operation is insertion or deletion in the middle. Java LinkedList class uses a doubly linked list to store the elements. It provides a linked-list data structure. It inherits the AbstractList class and implements List and Deque interfaces.



- Underlying data structure is doubly link list.
- Insertion order preserved
- Duplicates are allowed
- Heterogeneous and homogeneous objects are allowed
- Null insertion is allowed

Note: If our frequent operation is retrieval then linklist is the worst choice.

LinkedList:LinkList class we can be use to develop stack and queue. To develop stack and queue , lisnklist class provide six specific methods. These methods applicable only for linklist object. You cannot apply anywhere else.
These methods that support stack and queue implementation are:

void addFirst(Object o)
void addLast(Object o)
Object getFirst()
Object getLast()
Object removeFirst()
Object removeLast();
Constructors:
LinkedList l=new LinkedList();
LinkedList l= new LinkedList(Collection c);
Example: import java.util.*;
class LinkedListDemo{ public static void main(String[] args){
LinkedList l=new LinkedList();
l.add("Santosh");l.add(10);l.add(null);
l.set(0,"Mishra");//Replace 0th element with Mishra
l.add(0,"Venky");l.removeLast();
l.addFirst("CCC");
System.out.println(l);
}}

- Java LinkedList class can contain duplicate elements.
- Java LinkedList class maintains insertion order.
- Java LinkedList class is non synchronized.
- In Java LinkedList class, manipulation is fast because no shifting needs to occur.
- Java LinkedList class can be used as a list, stack or queue.

Java LinkedList Example

```
import java.util.*;
public class LinkedList1{
 public static void main(String args[]){

  LinkedList<String> al=new LinkedList<String>();
  al.add("Ravi");
  al.add("Vijay");
  al.add("Ravi");
  al.add("Ajay");

  Iterator<String> itr=al.iterator();
  while(itr.hasNext()){
   System.out.println(itr.next());
  }   }}
```

Java LinkedList example to add elements

```
import java.util.*;
public class LinkedList2{
 public static void main(String args[]){
 LinkedList<String> ll=new LinkedList<String>();
        System.out.println("Initial list of elements: "+ll);
        ll.add("Ravi");          ll.add("Vijay");          ll.add("Ajay");
        System.out.println("After invoking add(E e) method: "+ll);
        //Adding an element at the specific position
        ll.add(1, "Gaurav");          System.out.println("After invoking add(int index, E element) method: "+ll);
        LinkedList<String> ll2=new LinkedList<String>();
        ll2.add("Sonoo");          ll2.add("Hanumat");
        //Adding second list elements to the first list
        ll.addAll(ll2);
        System.out.println("After invoking addAll(Collection<? extends E> c) method: "+ll);
        LinkedList<String> ll3=new LinkedList<String>();
        ll3.add("John");          ll3.add("Rahul");
        //Adding second list elements to the first list at specific position
        ll.addAll(1, ll3);
        System.out.println("After invoking addAll(int index, Collection<? extends E> c) method: "+ll);
        //Adding an element at the first position
        ll.addFirst("Lokesh");          System.out.println("After invoking addFirst(E e) method: "+ll);
        //Adding an element at the last position
        ll.addLast("Harsh");          System.out.println("After invoking addLast(E e) method: "+ll);  }}
```

Java LinkedList example to remove elements

```
import java.util.*;
public class LinkedList3 {

    public static void main(String [] args)
    {
      LinkedList<String> ll=new LinkedList<String>();
      ll.add("Ravi"); ll.add("Vijay"); ll.add("Ajay");  ll.add("Anuj");  ll.add("Gaurav");  ll.add("Harsh");
      ll.add("Virat");  ll.add("Gaurav");  ll.add("Harsh");      ll.add("Amit");
      System.out.println("Initial list of elements: "+ll);
    //Removing specific element from arraylist
        ll.remove("Vijay");       System.out.println("After invoking remove(object) method: "+ll);
    //Removing element on the basis of specific position
        ll.remove(0);          System.out.println("After invoking remove(index) method: "+ll);
        LinkedList<String> ll2=new LinkedList<String>();
        ll2.add("Ravi");               ll2.add("Hanumat");
    // Adding new elements to arraylist
        ll.addAll(ll2);               System.out.println("Updated list : "+ll);
    //Removing all the new elements from arraylist
        ll.removeAll(ll2);               System.out.println("After invoking removeAll() method: "+ll);
    //Removing first element from the list
        ll.removeFirst();               System.out.println("After invoking removeFirst() method: "+ll);
     //Removing first element from the list
        ll.removeLast();        System.out.println("After invoking removeLast() method: "+ll);
     //Removing first occurrence of element from the list
```

Java LinkedList Example to reverse a list of elements

```
import java.util.*;
public class LinkedList4{
 public static void main(String args[]){

  LinkedList<String> ll=new LinkedList<String>();
        ll.add("Ravi");
        ll.add("Vijay");
        ll.add("Ajay");
        //Traversing the list of elements in reverse order
        Iterator i=ll.descendingIterator();
        while(i.hasNext())
        {
           System.out.println(i.next());
        }

 }
}
```

Java LinkedList Example: Book

```
import java.util.*;
class Book {
int id;
String name,author,publisher;
int quantity;
public Book(int id, String name, String author, String publisher, int quantity) {
    this.id = id;
    this.name = name;
    this.author = author;
    this.publisher = publisher;
    this.quantity = quantity;
}
}
public class LinkedListExample {
public static void main(String[] args) {
    //Creating list of Books
    List<Book> list=new LinkedList<Book>();
    //Creating Books
    Book b1=new Book(101,"Let us C","Yashwant Kanetkar","BPB",8);
    Book b2=new Book(102,"Data Communications & Networking","Forouzan","Mc Graw Hill",4);
    Book b3=new Book(103,"Operating System","Galvin","Wiley",6);
```

```
//Adding Books to list
    list.add(b1);
    list.add(b2);
    list.add(b3);
    //Traversing list
    for(Book b:list){
    System.out.println(b.id+" "+b.name+" "+b.author+" "+b.publisher+" "+b.quantity);
    }
}
}
```

Difference Between ArrayList and LinkedList

ArrayList	LinkedList
1) ArrayList internally uses a **dynamic array** to store the elements.	LinkedList internally uses a **doubly linked list** to store the elements.
2) Manipulation with ArrayList is **slow** because it internally uses an array. If any element is removed from the array, all the other elements are shifted in memory.	Manipulation with LinkedList is **faster** than ArrayList because it uses a doubly linked list, so no bit shifting is required in memory.
3) An ArrayList class can **act as a list** only because it implements List only.	LinkedList class can **act as a list and queue** both because it implements List and Deque interfaces.
4) ArrayList is **better for storing and accessing** data.	LinkedList is **better for manipulating** data.
5) The memory location for the elements of an ArrayList is contiguous.	The location for the elements of a linked list is not contagious.
6) Generally, when an ArrayList is initialized, a default capacity of 10 is assigned to the ArrayList.	There is no case of default capacity in a LinkedList. In LinkedList, an empty list is created when a LinkedList is initialized.
7) To be precise, an ArrayList is a resizable array.	LinkedList implements the doubly linked list of the list interface.

Vector:

Underlying data structure is Resizable/growable array.
Duplicates are allowed.
Insertion Order Preserved.
Null insertion allowed
Heterogeneous Objects are allowed
Serialization and clonable already implemented
RandomAccess interface already implemented.
Most of the method of vector are synchronized. Means Vector object is thread safe.

Constructor:

Vector()
Vector(int initialCapacity)
Vector(int initialCapacity, int capacityIncrement)
Vector(Collection c)

Example-1

```
import java.util.*;
public class VectorExample {
    public static void main(String args[]) {
      //Create a vector
      Vector<String> vec = new Vector<String>();
      //Adding elements using add() method of List
      vec.add("Tiger");
      vec.add("Lion");
      vec.add("Dog");
      vec.add("Elephant");
      //Adding elements using addElement() method of Vector
      vec.addElement("Rat");
      vec.addElement("Cat");
      vec.addElement("Deer");

      System.out.println("Elements are: "+vec);
    }
}
```

Example-2

```
import java.util.*;
public class VectorExample1 {
    public static void main(String args[]) {
      //Create an empty vector with initial capacity 4
      Vector<String> vec = new Vector<String>(4);
      //Adding elements to a vector
      vec.add("Tiger");
      vec.add("Lion");
      vec.add("Dog");
      vec.add("Elephant");
      //Check size and capacity
      System.out.println("Size is: "+vec.size());
      System.out.println("Default capacity is: "+vec.capacity());

      //Display Vector elements
      System.out.println("Vector element is: "+vec);
      vec.addElement("Rat");
      vec.addElement("Cat");
      vec.addElement("Deer");
```

```
//Again check size and capacity after two insertions
        System.out.println("Size after addition: "+vec.size());
System.out.println("Capacity after addition is: "+vec.capacity())
;          //Display Vector elements again
        System.out.println("Elements are: "+vec);
        //Checking if Tiger is present or not in this vector
         if(vec.contains("Tiger"))   {
           System.out.println("Tiger is present at the index " +vec
.indexOf("Tiger"));
         }
         else
         {
           System.out.println("Tiger is not present in the list.");
         }
         //Get the first element
        System.out.println("The first animal of the vector is = "+v
ec.firstElement());
        //Get the last element
        System.out.println("The last animal of the vector is = "+v
ec.lastElement());        }  }
```

Stack

The **stack** is a linear data structure that is used to store the collection of objects. It is based on **Last-In-First-Out** (LIFO). Java collection framework provides many interfaces and classes to store the collection of objects. One of them is the **Stack class** that provides different operations such as push, pop, search, etc.

The stack data structure has the two most important operations that are **push** and **pop**. The push operation inserts an element into the stack and pop operation removes an element from the top of the stack.

In Java, **Stack** is a class that falls under the Collection framework that extends the **Vector** class. It also implements interfaces **List, Collection, Iterable, Cloneable, Serializable.** It represents the LIFO stack of objects. Before using the Stack class, we must import the java.util package.

Creating a Stack

If we want to create a stack, first, import the java.util package and create an object of the Stack class.

1. Stack stk = **new** Stack();

Or

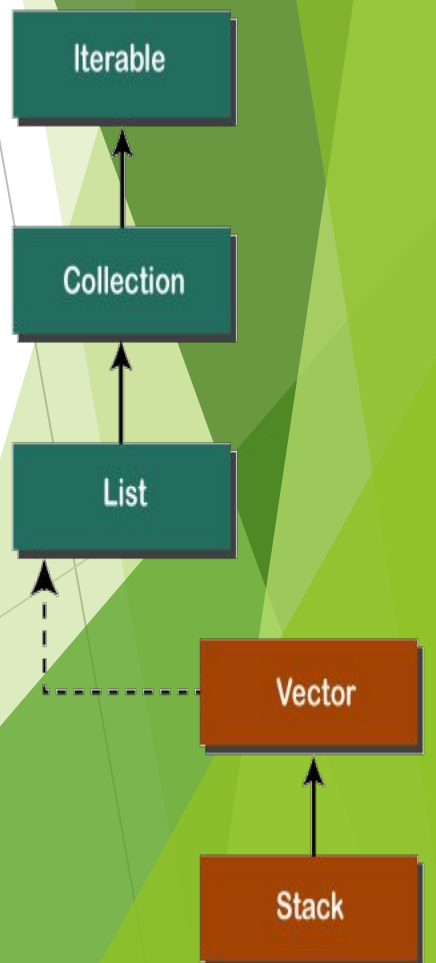
1. Stack<type> stk = **new** Stack<>();

We can perform push, pop, peek and search operation on the stack.

The Java Stack class provides mainly five methods to perform these operations.

Along with this, it also provides all the methods of the Java Vector class.

Method	Modifier and Type	Method Description
empty()	boolean	The method checks the stack is empty or not.
push(E item)	E	The method pushes (insert) an element onto the top of the stack.
pop()	E	The method removes an element from the top of the stack and returns the same element as the value of that function.
peek()	E	The method looks at the top element of the stack without removing it.
search(Object o)	int	The method searches the specified object and returns the position of the object.



Example-1

```
import java.util.Stack;
public class StackEmptyMethodExample
{
public static void main(String[] args)
{
//creating an instance of Stack class
Stack<Integer> stk= new Stack<>();
// checking stack is empty or not
boolean result = stk.empty();
System.out.println("Is the stack empty? " + result);
// pushing elements into stack
stk.push(78);
stk.push(113);
stk.push(90);
stk.push(120);
//prints elements of the stack
System.out.println("Elements in Stack: " + stk);
result = stk.empty();
System.out.println("Is the stack empty? " + result);
} }
```

Example-2 (Push and Pop)

```
import java.util.*;
public class StackPushPopExample
{
public static void main(String args[])
{
//creating an object of Stack class
Stack <Integer> stk = new Stack<>();
System.out.println("stack: " + stk);
//pushing elements into the stack
pushelmnt(stk, 20);
pushelmnt(stk, 13);
pushelmnt(stk, 89);
pushelmnt(stk, 90);
pushelmnt(stk, 11);
pushelmnt(stk, 45);
pushelmnt(stk, 18);
//popping elements from the stack
popelmnt(stk);
popelmnt(stk);
//throws exception if the stack is empty
```

```
try   {
popelmnt(stk);
}
catch (EmptyStackException e)   {
System.out.println("empty stack");
}  }
//performing push operation
static void pushelmnt(Stack stk, int x) {
//invoking push() method
stk.push(new Integer(x));
System.out.println("push -> " + x);
//prints modified stack
System.out.println("stack: " + stk);  }
//performing pop operation
static void popelmnt(Stack stk)   {
System.out.print("pop -> ");
//invoking pop() method
Integer x = (Integer) stk.pop();
System.out.println(x);
//prints modified stack
System.out.println("stack: " + stk);  }}
```

Stack Iteration

```
import java.util.Iterator;
import java.util.Stack;
public class StackIterationExample1
{
public static void main (String[] args)
{
//creating an object of Stack class
Stack stk = new Stack();
//pushing elements into stack
stk.push("BMW");
stk.push("Audi");
stk.push("Ferrari");
stk.push("Bugatti");
stk.push("Jaguar");
//iteration over the stack
Iterator iterator = stk.iterator();
while(iterator.hasNext())
{
Object values = iterator.next();
System.out.println(values);
}    }}
```

Sorting:

Comparable(I) is used for default natural sorting order.

Comparator(I) is used for Customized sorting order.

Comparator: Comparator interface contains on method given below

Compare(Object obj1,Object obj2)

Returns –ve value iff obj1 has to come before obj2

Returns +ve value iff obj1 has to come after obj2.

Returns 0 iff obj1 and obj2 are equal.

Sorting Without Comparator:

```
import java.util.*;
class MyComparator{
public static void main(String[] args){
ArrayList<Integer> l=new ArrayList<Integer>();
l.add(20); l.add(10); l.add(25); l.add(5); l.add(30); l.add(0); l.add(15);
System.out.println(l);
Collections.sort(l);
System.out.println(l);
}}
```

EXAMPLE-2

```
import java.util.*;
import java.io.*;
class Student implements Comparable
  {
     int rollno;
     String name;
     int age;

     Student(int rollno,String name,int age)
     {
     this.rollno=rollno;
     this.name=name;
     this.age=age;
     }
```

```
 public int compareTo(Student st){
        if(age==st.age)
            return 0;
        else if(age>st.age)
            return 1;
        else
            return -1;
}
}
class Main{
public static void main(String args[]){
ArrayList al=new ArrayList();
al.add(new Student(256,"Atul",56));
al.add(new Student(289,"Aquib",34));
al.add(new Student(198,"Jaya",61));

Collections.sort(al);
for(Student st:al){
System.out.println(st.rollno+" "+st.name+" "+st.age);
}
}
}
```

Sorting With Comparator:

```
import java.util.*;
class MyComparator implements Comparator<Integer>{
public int compare(Integer I1,Integer I2){
if(I1<I2){
return -1;}
else if (I1>I2){
return 1;}
else
{
return 0;}
}}
class Test{
public static void main(String[] args){
ArrayList<Integer> l=new ArrayList<Integer>();
l.add(20); l.add(10); l.add(25); l.add(5); l.add(30); l.add(0); l.add(15);
System.out.println(l);
Collections.sort(l,new MyComparator());
System.out.println(l);
}}
```

Example-2

```
import java.io.*;
import java.lang.*;
import java.util.*;
class Student {
    int rollno;
    String name, address;
    public Student(int rollno, String name, String address)
    {
        this.rollno = rollno;
        this.name = name;
        this.address = address;
    }
    public String toString()
    {
        return this.rollno + " " + this.name + " "
            + this.address;
    }
}
```

```
class Sortbyroll implements Comparator {

    public int compare(Student a, Student b)
    {

        return a.rollno - b.rollno;
    }
}


class Sortbyname implements Comparator {

    public int compare(Student a, Student b)
    {

        return a.name.compareTo(b.name);
    }
}
```

```
class Main {

    public static void main(String[] args)
    {

        ArrayList ar = new ArrayList();
        ar.add(new Student(234, "Anshuman", "Lucknow"));
        ar.add(new Student(345, "Atul", "Noida"));
        ar.add(new Student(734, "Aquib", "Shilong"));
        ar.add(new Student(563, "Jaya", "Delhi"));
        System.out.println("Unsorted");
        for (int i = 0; i < ar.size(); i++)
            System.out.println(ar.get(i));
        Collections.sort(ar, new Sortbyroll());
        System.out.println("\nSorted by rollno");
        for (int i = 0; i < ar.size(); i++)
            System.out.println(ar.get(i));
        Collections.sort(ar, new Sortbyname());
        System.out.println("\nSorted by name");
        for (int i = 0; i < ar.size(); i++)
            System.out.println(ar.get(i));
    }
}
```

Key differences between comparable vs comparator

Here are some of the key differences between comparable vs comparator:

- The comparable interface uses the compareTo() method to sort, whereas the comparator interface uses the compare() method.
- A comparable interface can be found in java.lang package, whereas the comparator interface, can be found in java.util package.
- A comparable interface modifies the actual or the original class. But, the comparator interface does not modify the actual or the original class.
- The comparable interface sorts the data according to natural sorting order, whereas the comparator interface sorts the data according to customized sorting order.

Set

An unordered collection or list in which duplicates are not allowed is referred to as a **collection interface**. The set interface is used to create the mathematical set. The set interface use collection interface's methods to avoid the insertion of the same elements. **SortedSet** and **NavigableSet** are two interfaces that extend the set implementation.

the **NavigableSet** and **SortedSet** are both the interfaces. The **NavigableSet** extends the SortedSet, so it will not retain the insertion order and store the data in a sorted way. SetExample1.java

```
import java.util.*;
public class setExample{
   public static void main(String[] args)
   {
      // creating LinkedHashSet using the Set
      Set<String> data = new LinkedHashSet<String>();

      data.add("JavaTpoint");
      data.add("Set");
      data.add("Example");
      data.add("Set");

      System.out.println(data);
   }
}
```

Operations on the Set Interface

On the Set, we can perform all the basic mathematical operations like intersection, union and difference. Suppose, we have two sets, i.e., set1 = [22, 45, 33, 66, 55, 34, 77] and set2 = [33, 2, 83, 45, 3, 12, 55]. We can perform the following operation on the Set:

- **Intersection:** The intersection operation returns all those elements which are present in both the set. The intersection of set1 and set2 will be [33, 45, 55].
- **Union:** The union operation returns all the elements of set1 and set2 in a single set, and that set can either be set1 or set2. The union of set1 and set2 will be [2, 3, 12, 22, 33, 34, 45, 55, 66, 77, 83].
- **Difference:** The difference operation deletes the values from the set which are present in another set. The difference of the set1 and set2 will be [66, 34, 22, 77].

In set, **addAll()** method is used to perform the union, **retainAll()** method is used to perform the intersection and **removeAll()** method is used to perform difference.

SetExample2.java

```
import java.util.*;
public class SetOperations   {
   public static void main(String args[])       {
      Integer[] A = {22, 45,33, 66, 55, 34, 77};
      Integer[] B = {33, 2, 83, 45, 3, 12, 55};
      Set<Integer> set1 = new HashSet<Integer>();
      set1.addAll(Arrays.asList(A));
      Set<Integer> set2 = new HashSet<Integer>();
      set2.addAll(Arrays.asList(B));
```

```
// Finding Union of set1 and set2
    Set<Integer> union_data = new HashSet<Integer>(set1);
    union_data.addAll(set2);
    System.out.print("Union of set1 and set2 is:");
    System.out.println(union_data);

    // Finding Intersection of set1 and set2
    Set<Integer> intersection_data = new HashSet<Integer>(set1);
    intersection_data.retainAll(set2);
    System.out.print("Intersection of set1 and set2 is:");
    System.out.println(intersection_data);

    // Finding Difference of set1 and set2
    Set<Integer> difference_data = new HashSet<Integer>(set1);
    difference_data.removeAll(set2);
    System.out.print("Difference of set1 and set2 is:");
    System.out.println(difference_data);
}
```

Set Methods

There are several methods available in the set interface which we can use to perform a certain operation on our sets. These methods are as follows:

1) add()

The add() method insert a new value to the set. The method returns true and false depending on the presence of the insertion element. It returns false if the element is already present in the set and returns true if it is not present in the set.

Syntax:

```
boolean add(type element).
```

SetExample3.java

```
import java.io.*;
import java.util.*;
public class addMethod {
   public static void main(String args[])      {
      Set<Integer> data = new LinkedHashSet<Integer>();
      data.add(31);
      data.add(21);
      data.add(41);
      data.add(11);
      data.add(61);
      data.add(51);
      System.out.println("data: " + data);      }}
```

2) addAll()

The addAll() method appends all the elements of the specified collection to the set.

Syntax:

```
boolean addAll(Collection data)
```

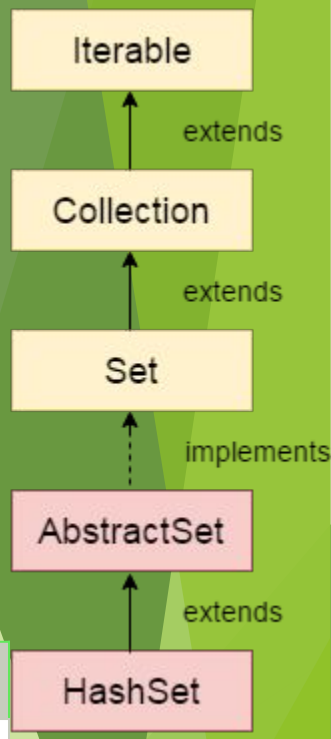
SetExample4.java

```
import java.io.*;
import java.util.*;
class addAllMethod {
   public static void main(String args[])
   {
      Set<Integer> data = new LinkedHashSet<Integer>();
      data.add(31);
      data.add(21);
      data.add(41);
      System.out.println("Set: " + data);
      ArrayList<Integer> newData = new ArrayList<Integer>();
      newData.add(91);
      newData.add(71);
      newData.add(81);
      data.addAll(newData);
      System.out.println("Set: " + data);      }   }
```

HashSet:

Java HashSet class is used to create a collection that uses a hash table for storage. It inherits the AbstractSet class and implements Set interface.

- HashSet stores the elements by using a mechanism called **hashing.**
- HashSet contains unique elements only.
- HashSet allows null value.
- HashSet class is non synchronized.
- HashSet doesn't maintain the insertion order. Here, elements are inserted on the basis of their hashcode.
- HashSet is the best approach for search operations.
- The initial default capacity of HashSet is 16, and the load factor is 0.75.



Constructors of Java HashSet class

SN	Constructor	Description
1)	HashSet()	It is used to construct a default HashSet.
2)	HashSet(int capacity)	It is used to initialize the capacity of the hash set to the given integer value capacity. The capacity grows automatically as elements are added to the HashSet.
3)	HashSet(int capacity, float loadFactor)	It is used to initialize the capacity of the hash set to the given integer value capacity and the specified load factor.
4)	HashSet(Collection<? extends E> c)	It is used to initialize the hash set by using the elements of the collection c.

Example-1

```
import java.util.*;
class HashSet1{
 public static void main(String args[]){
  //Creating HashSet and adding elements
    HashSet<String> set=new HashSet();
          set.add("One");
          set.add("Two");
          set.add("Three");
          set.add("Four");
          set.add("Five");
          Iterator<String> i=set.iterator();
          while(i.hasNext())
          {
          System.out.println(i.next());
          }
 }
}
```

Example-2

```
import java.util.*;
class HashSet2{
 public static void main(String args[]){
  //Creating HashSet and adding elements
  HashSet<String> set=new HashSet<String>();
  set.add("Ravi");
  set.add("Vijay");
  set.add("Ravi");
  set.add("Ajay");
  //Traversing elements
  Iterator<String> itr=set.iterator();
  while(itr.hasNext()){
   System.out.println(itr.next());
  }
 }
}
```

Example-3

```
import java.util.*;
class Book {
int id;
String name,author,publisher;
int quantity;
public Book(int id, String name, String author, String publisher, int quantity) {
    this.id = id;
    this.name = name;
    this.author = author;
    this.publisher = publisher;
    this.quantity = quantity;
}
}
public class HashSetExample {
public static void main(String[] args) {
    HashSet<Book> set=new HashSet<Book>();
```

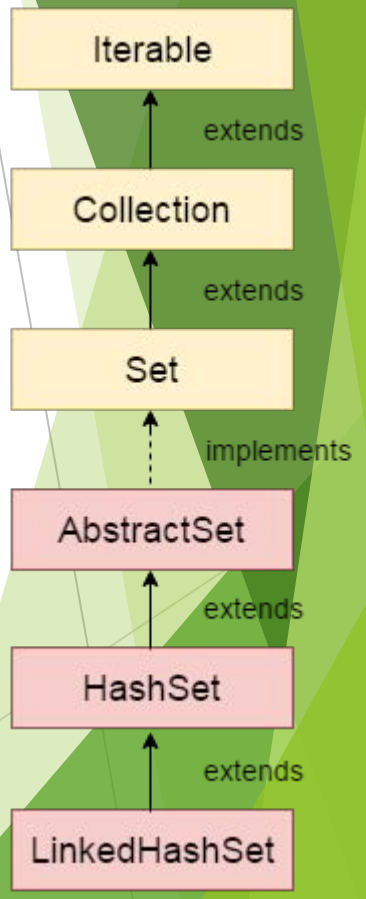
Example-3(Continue…)

```
//Creating Books
   Book b1=new Book(101,"Let us C","Yashwant Kanetkar","BPB",8);
   Book b2=new Book(102,"Data Communications & Networking","Forouzan","Mc
Graw Hill",4);
   Book b3=new Book(103,"Operating System","Galvin","Wiley",6);
   //Adding Books to HashSet
   set.add(b1);
   set.add(b2);
   set.add(b3);
   //Traversing HashSet
   for(Book b:set){
   System.out.println(b.id+" "+b.name+" "+b.author+" "+b.publisher+" "+b.quantity);

   }
}
}
```

LinkedHashSet Class

Java LinkedHashSet class is a Hashtable and Linked list implementation of the Set interface. It inherits the HashSet class and implements the Set interface.

- Java LinkedHashSet class contains unique elements only like HashSet.
- Java LinkedHashSet class provides all optional set operations and permits null elements.
- Java LinkedHashSet class is non-synchronized.
- Java LinkedHashSet class maintains insertion order.



Constructors of Java LinkedHashSet Class

Constructor	Description
HashSet()	It is used to construct a default HashSet.
HashSet(Collection c)	It is used to initialize the hash set by using the elements of the collection c.
LinkedHashSet(int capacity)	It is used to initialize the capacity of the linked hash set to the given integer value capacity.
LinkedHashSet(int capacity, float fillRatio)	It is used to initialize both the capacity and the fill ratio (also called load capacity) of the hash set from its argument.

Example-1

```
import java.util.*;
class LinkedHashSet1{
 public static void main(String args[]){
 //Creating HashSet and adding elements
        LinkedHashSet<String> set=new LinkedHashSet();
                set.add("One");
                set.add("Two");
                set.add("Three");
                set.add("Four");
                set.add("Five");
                Iterator<String> i=set.iterator();
                while(i.hasNext())
                {
                System.out.println(i.next());
                }
 }
}
```

Example-2

```
import java.util.*;
class LinkedHashSet2{
 public static void main(String args[]){
  LinkedHashSet<String> al=new LinkedHashSet<String>();
  al.add("Ravi");
  al.add("Vijay");
  al.add("Ravi");
  al.add("Ajay");
  Iterator<String> itr=al.iterator();
  while(itr.hasNext()){
   System.out.println(itr.next());
  }
 }
}
```

Example-3

```
import java.util.*;
class Book {
int id;
String name,author,publisher;
int quantity;
public Book(int id, String name, String author, String publisher, int quantity) {
    this.id = id;
    this.name = name;
    this.author = author;
    this.publisher = publisher;
    this.quantity = quantity;
}
}
public class LinkedHashSetExample {
public static void main(String[] args) {
    LinkedHashSet<Book> hs=new LinkedHashSet<Book>();
    //Creating Books
```

Example-3

```
 Book b1=new Book(101,"Let us C","Yashwant Kanetkar","BPB",8);
   Book b2=new Book(102,"Data Communications & Networking","Forouzan","Mc
Graw Hill",4);
   Book b3=new Book(103,"Operating System","Galvin","Wiley",6);
   //Adding Books to hash table
   hs.add(b1);
   hs.add(b2);
   hs.add(b3);
   //Traversing hash table
   for(Book b:hs){
   System.out.println(b.id+" "+b.name+" "+b.author+" "+b.publisher+" "+b.quantity);

   }
}
}
```

TreeSet

Java TreeSet class implements the Set interface that uses a tree for storage. It inherits AbstractSet class and implements the NavigableSet interface. The objects of the TreeSet class are stored in ascending order.

The important points about the Java TreeSet class are:

- Java TreeSet class contains unique elements only like HashSet.
- Java TreeSet class access and retrieval times are quiet fast.
- Java TreeSet class doesn't allow null element.
- Java TreeSet class is non synchronized.
- Java TreeSet class maintains ascending order.
- The TreeSet can only allow those generic types that are comparable.

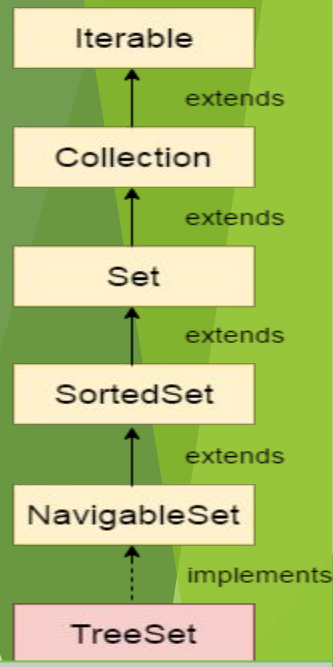
Internal Working of The TreeSet Class

TreeSet is being implemented using a binary search tree, which is self-balancing just like a Red-Black Tree. Therefore, operations such as a search, remove, and add consume O(log(N)) time. The reason behind this is there in the self-balancing tree. It is there to ensure that the tree height never exceeds O(log(N)) for all of the mentioned operations. Therefore, it is one of the efficient data structures in order to keep the large data that is sorted and also to do operations on it.

Synchronization of The TreeSet Class

As already mentioned above, the TreeSet class is not synchronized. It means if more than one thread concurrently accesses a tree set, and one of the accessing threads modify it, then the synchronization must be done manually. It is usually done by doing some object synchronization that encapsulates the set. However, in the case where no such object is found, then the set must be wrapped with the help of the Collections.synchronizedSet() method. It is advised to use the method during creation time in order to avoid the unsynchronized access of the set. The following code snippet shows the same.

```
TreeSet treeSet = new TreeSet();
Set syncrSet = Collections.synchronziedSet(treeSet);
```



Constructors of Java TreeSet Class

Constructor	Description
TreeSet()	It is used to construct an empty tree set that will be sorted in ascending order according to the natural order of the tree set.
TreeSet(Collection<? extends E> c)	It is used to build a new tree set that contains the elements of the collection c.
TreeSet(Comparator<? super E> comparator)	It is used to construct an empty tree set that will be sorted according to given comparator.
TreeSet(SortedSet<E> s)	It is used to build a TreeSet that contains the elements of the given SortedSet.

Example-1

```
 import java.util.*;
class TreeSet1{
 public static void main(String args[]){
  //Creating and adding elements
  TreeSet<String> al=new TreeSet<String>();
  al.add("Ravi");
  al.add("Vijay");
  al.add("Ravi");
  al.add("Ajay");
  //Traversing elements
  Iterator<String> itr=al.iterator();
  while(itr.hasNext()){
   System.out.println(itr.next());
  }
 }
}
```

Example-2

```
 import java.util.*;
class TreeSet2{
 public static void main(String args[]){
 TreeSet<String> set=new TreeSet<String>();
        set.add("Ravi");
        set.add("Vijay");
        set.add("Ajay");
        System.out.println("Traversing element through Iterator in descending order");
        Iterator i=set.descendingIterator();
        while(i.hasNext())
        {
           System.out.println(i.next());
        }

 }
}
```

E pollFirst()	It is used to retrieve and remove the lowest(first) element.
E pollLast()	It is used to retrieve and remove the highest(last) element.

Example-3

```
import java.util.*;
class TreeSet3{
public static void main(String args[]){
TreeSet<Integer> set=new TreeSet<Integer>();
        set.add(24);
        set.add(66);
        set.add(12);
        set.add(15);
        System.out.println("Lowest Value: "+set.pollFirst());
        System.out.println("Highest Value: "+set.pollLast());
}
}
```

NavigableSet descendingSet()	It returns the elements in reverse order.
SortedSet headSet(E toElement)	It returns the group of elements that are less than the specified element.
NavigableSet headSet(E toElement, boolean inclusive)	It returns the group of elements that are less than or equal to(if, inclusive is true) the specified element.
NavigableSet subSet(E fromElement, boolean fromInclusive, E toElement, boolean toInclusive)	It returns a set of elements that lie between the given range.
SortedSet subSet(E fromElement, E toElement))	It returns a set of elements that lie between the given range which includes fromElement and excludes toElement.
SortedSet tailSet(E fromElement)	It returns a set of elements that are greater than or equal to the specified element.
NavigableSet tailSet(E fromElement, boolean inclusive)	It returns a set of elements that are greater than or equal to (if, inclusive is true) the specified element.

Example-5

Java TreeSet Example: Book

Let's see a TreeSet example where we are adding books to the set and printing all the books. The elements in TreeSet must be of a Comparable type. String and Wrapper classes are Comparable by default. To add user-defined objects in TreeSet, you need to implement the Comparable interface.

```
import java.util.*;
class Book implements Comparable<Book>{
int id;
String name,author,publisher;
int quantity;
public Book(int id, String name, String author, String publisher, int quantity) {
    this.id = id;
    this.name = name;
    this.author = author;
    this.publisher = publisher;
    this.quantity = quantity;
}
```

```
// implementing the abstract method
public int compareTo(Book b) {
    if(id>b.id){
        return 1;
    }else if(id<b.id){
        return -1;
    }else{
    return 0;
    }  }}
public class TreeSetExample {
public static void main(String[] args) {
    Set<Book> set=new TreeSet<Book>();
    //Creating Books
    Book b1=new Book(121,"Let us C","Yashwant Kanetkar","BPB",8);
    Book b2=new Book(233,"Operating System","Galvin","Wiley",6);
    Book b3=new Book(101,"Data Communications & Networking","Forouzan","Mc
Graw Hill",4);
    //Adding Books to TreeSet
    set.add(b1);        set.add(b2);        set.add(b3);
    //Traversing TreeSet
    for(Book b:set){
    System.out.println(b.id+" "+b.name+" "+b.author+" "+b.publisher+" "+b.quantity);
      }    }}
```

ClassCast Exception in TreeSet

If we add an object of the class that is not implementing the Comparable interface, the ClassCast Exception is raised. Observe the following program.

```
// important import statement
import java.util.*;
class Employee {
int empId;
String name;

// getting the name of the employee
String getName()
{
    return this.name;
}

// setting the name of the employee
void setName(String name)
{
this.name = name;
}
```

```
// setting the employee id
// of the employee
void setId(int a)  {
this.empId = a;  }
// retrieving the employee id of
// the employee
int getId()  {
return this.empId;  }  }
public class ClassCastExceptionTreeSet  {
// main method
public static void main(String[] argvs)  {
// creating objects of the class Employee
Employee obj1 = new Employee();
Employee obj2 = new Employee();
TreeSet<Employee> ts =  new TreeSet<Employee>();
// adding the employee objects to
// the TreeSet class
ts.add(obj1);  ts.add(obj2);
System.out.println("The program has been executed successfull
y.");  }  }
```

When we compile the above program, we get the ClassCastException, as shown below.

```
Exception in thread "main" java.lang.ClassCastException: class Employee cannot be cast to
class java.lang.Comparable (Employee is in unnamed module of
 loader 'app'; java.lang.Comparable is in module java.base of loader 'bootstrap’)
at java.base/java.util.TreeMap.compare(TreeMap.java:1569)
at java.base/java.util.TreeMap.addEntryToEmptyMap(TreeMap.java:776)
 at java.base/java.util.TreeMap.put(TreeMap.java:785) at
java.base/java.util.TreeMap.put(TreeMap.java:534)
 at java.base/java.util.TreeSet.add(TreeSet.java:255) at
ClassCastExceptionTreeSet.main(ClassCastExceptionTreeSet.java:52)
```

Java Queue Interface

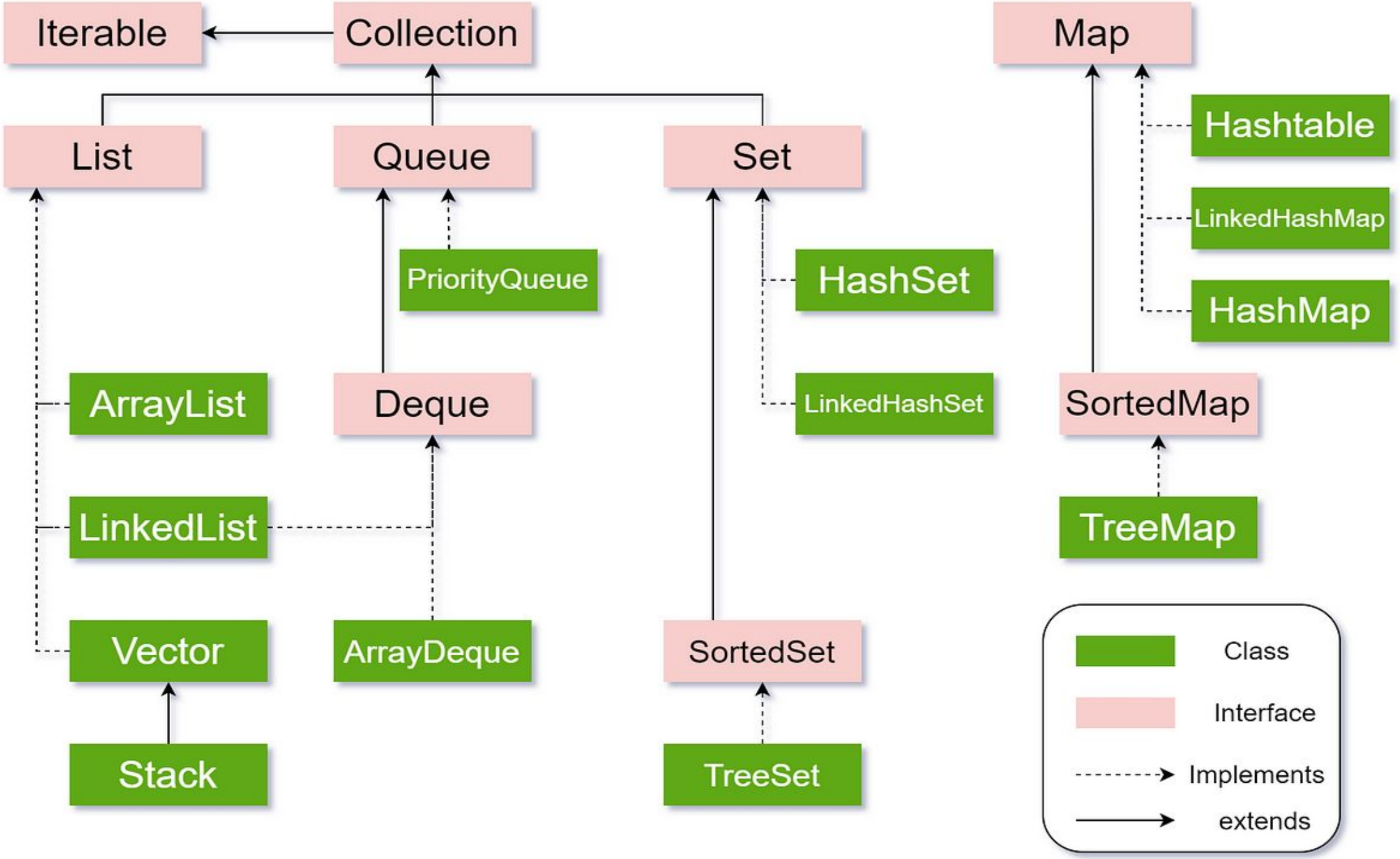
Queue is used to keep the elements that are processed in the First In First Out (FIFO) manner. It is an ordered list of objects, where insertion of elements occurs at the end of the list, and removal of elements occur at the beginning of the list.

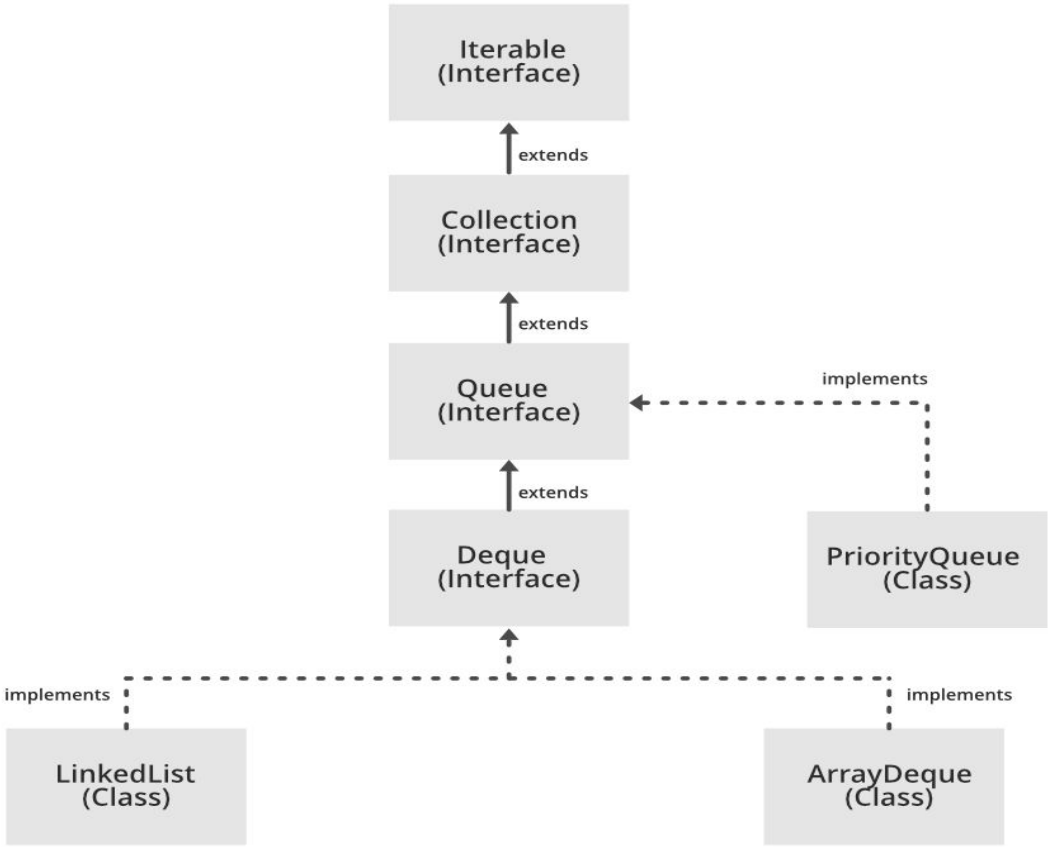
Features of a Queue

- FIFO concept is used for insertion and deletion of elements from a queue.
- The Java Queue provides support for all of the methods of the Collection interface including deletion, insertion, etc.
- PriorityQueue, ArrayBlockingQueue and LinkedList are the implementations that are used most frequently.
- The NullPointerException is raised, if any null operation is done on the BlockingQueues.
- Those Queues that are present in the *util* package are known as Unbounded Queues.
- Those Queues that are present in the *util.concurrent* package are known as bounded Queues.
- All Queues barring the Deques facilitates removal and insertion at the head and tail of the queue; respectively. In fact, deques support element insertion and removal at both ends.

Methods of Java Queue Interface

Method	Description
boolean add(object)	It is used to insert the specified element into this queue and return true upon success. Give exception if full.
boolean offer(object)	It is used to insert the specified element into this queue.
Object remove()	It is used to retrieves and removes the head of this queue.
Object poll()	It is used to retrieves and removes the head of this queue, or returns null if this queue is empty.
Object element()	It is used to retrieves, but does not remove, the head of this queue.
Object peek()	It is used to retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.


Iterable
Collection
Map
List
Queue
Set
Hashtable
LinkedHashMap
HashMap
PriorityQueue
HashSet
ArrayList
Deque
LinkedHashSet
SortedMap
LinkedList
TreeMap
Vector
ArrayDeque
SortedSet
Stack
TreeSet
Class
Interface
Implements
extends


Iterable
(Interface)
extends
Collection
(Interface)
extends
Queue
(Interface)
implements
extends
Deque
(Interface)
PriorityQueue
(Class)
implements
implements
LinkedList
(Class)
ArrayDeque
(Class)

ArrayDeque in Java

The ArrayDeque in Java provides a way to apply resizable-array in addition to the implementation of the Deque interface. It is also known as Array Double Ended Queue or Array Deck. This is a special kind of array that grows and allows users to add or remove an element from both sides of the queue.

The ArrayDeque class in Java is an implementation of the Deque interface that uses a resizable array to store its elements. This class provides a more efficient alternative to the traditional Stack class, which was previously used for double-ended operations. The ArrayDeque class provides constant-time performance for inserting and removing elements from both ends of the queue, making it a good choice for scenarios where you need to perform many add and remove operations.A few important points on Priority Queue are as follows:

- Advantages of using ArrayDeque:
- Efficient: The ArrayDeque class provides constant-time performance for inserting and removing elements from both ends of the queue, making it a good choice for scenarios where you need to perform many add and remove operations.
- Resizable: The ArrayDeque class uses a resizable array to store its elements, which means that it can grow and shrink dynamically to accommodate the number of elements in the queue.
- Lightweight: The ArrayDeque class is a lightweight data structure that does not require additional overhead, such as linked list nodes, making it a good choice for scenarios where memory is limited.
- Thread-safe: The ArrayDeque class is not thread-safe, but you can use the Collections.synchronizedDeque method to create a thread-safe version of the ArrayDeque class.
- Disadvantages of using ArrayDeque:
- Not synchronized: By default, the ArrayDeque class is not synchronized, which means that multiple threads can access it simultaneously, leading to potential data corruption.
- Limited capacity: Although the ArrayDeque class uses a resizable array to store its elements, it still has a limited capacity, which means that you may need to create a new ArrayDeque when the old one reaches its maximum size.

ArrayDequeEample-1

```
import java.util.ArrayDeque;
import java.util.Deque;

public class Example {
public static void main(String[] args) {
    Deque<Integer> deque = new ArrayDeque<>();
    deque.addFirst(1);
    deque.addLast(2);
    int first = deque.removeFirst();
    int last = deque.removeLast();
    System.out.println("First: " + first + ", Last: " + last);}}
```

Output: First: 1, Last: 2

ArrayDeque Methods

METHOD	DESCRIPTION
add(Element e)	The method inserts a particular element at the end of the deque.
addAll(Collection<? extends E> c)	Adds all of the elements in the specified collection at the end of this deque, as if by calling addLast(E) on each one, in the order that they are returned by the collection's iterator.
addFirst(Element e)	The method inserts particular element at the start of the deque.
addLast(Element e)	The method inserts a particular element at the end of the deque. It is similar to the add() method
clear()	The method removes all deque elements.
clone()	The method copies the deque.
contains(Obj)	The method checks whether a deque contains the element or not
element()	The method returns element at the head of the deque
forEach(Consumer<? super E> action)	Performs the given action for each element of the Iterable until all elements have been processed or the action throws an exception.
getFirst()	The method returns first element of the deque
getLast()	The method returns last element of the deque
isEmpty()	The method checks whether the deque is empty or not.

iterator()	Returns an iterator over the elements in this deque.
offer(Element e)	The method inserts element at the end of deque.
offerFirst(Element e)	The method inserts element at the front of deque.
offerLast(Element e)	The method inserts element at the end of the deque.
peek()	The method returns head element without removing it.
poll()	The method returns head element and also removes it
pop()	The method pops out an element for stack represented by deque
push(Element e)	The method pushes an element onto stack represented by deque
remove()	The method returns head element and also removes it
remove(Object o)	Removes a single instance of the specified element from this deque.
removeAll(Collection<?> c)	Removes all of this collection's elements that are also contained in the specified collection (optional operation).
removeFirst()	The method returns the first element and also removes it
removeFirstOccurrence(Object o)	Removes the first occurrence of the specified element in this deque (when traversing the deque from head to tail).

removeIf(Predicate<? super Element> filter)	Removes all of the elements of this collection that satisfy the given predicate.
removeLast()	The method returns the last element and also removes it
removeLastOccurrence(Object o)	Removes the last occurrence of the specified element in this deque (when traversing the deque from head to tail).
retainAll(Collection<?> c)	Retains only the elements in this collection that are contained in the specified collection (optional operation).
size()	Returns the number of elements in this deque.
spliterator()	Creates a late-binding and fail-fast Spliterator over the elements in this deque.
toArray()	Returns an array containing all of the elements in this deque in proper sequence (from first to the last element).
toArray(T[] a)	Returns an array containing all of the elements in this deque in proper sequence (from first to the last element); the runtime type of the returned array is that of the specified array.

ArrayDequeEample-2

```
// Java program to Illustrate Addition of elements in ArrayDeque
import java.io.*;
import java.util.*;
// AddingElementsToArrayDeque
public class GFG {
        public static void main(String[] args)     {
        // Initializing a deque
        // since deque is an interface
        // it is assigned the
        // ArrayDeque class
        Deque<String> dq = new ArrayDeque<String>();
        // add() method to insert
        dq.add("The");       dq.addFirst("To");          dq.addLast("Geeks");
        // offer() method to insert
        dq.offer("For");             dq.offerFirst("Welcome");             dq.offerLast("Geeks");
        // Printing Elements of ArrayDeque to the console
        System.out.println("ArrayDeque : " + dq);     }}
```

ArrayDequeEample-3

```
// Java program to Access Elements of ArrayDeque Importing required classes
import java.io.*;
import java.util.*;
public class GFG {
	public static void main(String args[])	{
		ArrayDeque<String> de_que = new ArrayDeque<String>();
		de_que.add("Welcome");		de_que.add("To");		de_que.add("Geeks");
	de_que.add("4");
		de_que.add("Geeks");
		System.out.println("ArrayDeque: " + de_que);
		// Displaying the First element
		System.out.println("The first element is: "
				+ de_que.getFirst());

		// Displaying the Last element
		System.out.println("The last element is: "
				+ de_que.getLast());
	}
}
```

ArrayDequeEample-4

```
// Java program to Illustrate Removal Elements in Deque
import java.util.*;
public class GFG {
        public static void main(String[] args)  {
        Deque<String> dq = new ArrayDeque<String>();
        dq.add("One");
        // addFirst inserts at the front
        dq.addFirst("Two");
        // addLast inserts at the back
        dq.addLast("Three");
        // print elements to the console
        System.out.println("ArrayDeque : " + dq);
        // remove element as a stack from top/front
        System.out.println(dq.pop());
        // remove element as a queue from front
        System.out.println(dq.poll());
        // remove element from front
        System.out.println(dq.pollFirst());
        // remove element from back
        System.out.println(dq.pollLast());}}
```

ArrayDequeEample-5

```
// Java program to Illustrate Iteration of Elements in Deque
import java.util.*;
public class GFG {  public static void main(String[] args)     {
                        Deque<String> dq = new ArrayDeque<String>();
                    dq.add("For");
            // Adding element at the front using addFirst() method
            dq.addFirst("Geeks");
            // add element at the last using addLast() method
            dq.addLast("Geeks");            dq.add("is so good");
            // Iterate using Iterator interface from the front of the queue
            for (Iterator itr = dq.iterator(); itr.hasNext();) {
                    // Print the elements
                    System.out.print(itr.next() + " ");           }
                        System.out.println();
            // Iterate in reverse sequence in a queue
            for (Iterator itr = dq.descendingIterator();
                    itr.hasNext();) {
                    System.out.print(itr.next() + " ");           }      }}
```

Priority Queue

A PriorityQueue is used when the objects are supposed to be processed based on the priority. It is known that a Queue follows the First-In-First-Out algorithm, but sometimes the elements of the queue are needed to be processed according to the priority, that's when the PriorityQueue comes into play.

The PriorityQueue is based on the priority heap. The elements of the priority queue are ordered according to the natural ordering, or by a Comparator provided at queue construction time, depending on which constructor is used.

Features of a Queue

- A few important points on Priority Queue are as follows:
- PriorityQueue doesn't permit null.
- We can't create a PriorityQueue of Objects that are non-comparable
- PriorityQueue are unbound queues.
- The head of this queue is the least element with respect to the specified ordering. If multiple elements are tied for the least value, the head is one of those elements — ties are broken arbitrarily.
- Since PriorityQueue is not thread-safe, java provides PriorityBlockingQueue class that implements the BlockingQueue interface to use in a java multithreading environment.
- The queue retrieval operations poll, remove, peek, and element access the element at the head of the queue.
- It provides O(log(n)) time for add and poll methods.
- It inherits methods from AbstractQueue, AbstractCollection, Collection, and Object class.

Priority Queue Constructors:

1. PriorityQueue(): Creates a PriorityQueue with the default initial capacity (11) that orders its elements according to their natural ordering.
2. PriorityQueue(Collection<E> c): Creates a PriorityQueue containing the elements in the specified collection.
3. PriorityQueue(int initialCapacity): Creates a PriorityQueue with the specified initial capacity that orders its elements according to their natural ordering.
4. PriorityQueue(int initialCapacity, Comparator<E> comparator): Creates a PriorityQueue with the specified initial capacity that orders its elements according to the specified comparator.
5. PriorityQueue(PriorityQueue<E> c): Creates a PriorityQueue containing the elements in the specified priority queue.
6. Creates a PriorityQueue containing the elements in the specified sorted set.
7. PriorityQueue(Comparator<E> comparator): Creates a PriorityQueue with the default initial capacity and whose elements are ordered according to the specified comparator.

Priority Queue Constructors:

1. PriorityQueue(): Creates a PriorityQueue with the default initial capacity (11) that orders its elements according to their natural ordering.
2. PriorityQueue(Collection<E> c): Creates a PriorityQueue containing the elements in the specified collection.
3. PriorityQueue(int initialCapacity): Creates a PriorityQueue with the specified initial capacity that orders its elements according to their natural ordering.
4. PriorityQueue(int initialCapacity, Comparator<E> comparator): Creates a PriorityQueue with the specified initial capacity that orders its elements according to the specified comparator.
5. PriorityQueue(PriorityQueue<E> c): Creates a PriorityQueue containing the elements in the specified priority queue.
6. Creates a PriorityQueue containing the elements in the specified sorted set.
7. PriorityQueue(Comparator<E> comparator): Creates a PriorityQueue with the default initial capacity and whose elements are ordered according to the specified comparator.

Basic operations of the priority queue:

1. boolean add(E element): This method inserts the specified element into this priority queue.
2. public peek(): This method retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.
3. public poll(): This method retrieves and removes the head of this queue, or returns null if this queue is empty.

Priority Queue Constructors:

1. PriorityQueue(): Creates a PriorityQueue with the default initial capacity (11) that orders its elements according to their natural ordering.
2. PriorityQueue(Collection<E> c): Creates a PriorityQueue containing the elements in the specified collection.
3. PriorityQueue(int initialCapacity): Creates a PriorityQueue with the specified initial capacity that orders its elements according to their natural ordering.
4. PriorityQueue(int initialCapacity, Comparator<E> comparator): Creates a PriorityQueue with the specified initial capacity that orders its elements according to the specified comparator.
5. PriorityQueue(PriorityQueue<E> c): Creates a PriorityQueue containing the elements in the specified priority queue.
6. Creates a PriorityQueue containing the elements in the specified sorted set.
7. PriorityQueue(Comparator<E> comparator): Creates a PriorityQueue with the default initial capacity and whose elements are ordered according to the specified comparator.

Basic operations of the priority queue:

1. boolean add(E element): This method inserts the specified element into this priority queue.
2. public peek(): This method retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.
3. public poll(): This method retrieves and removes the head of this queue, or returns null if this queue is empty.

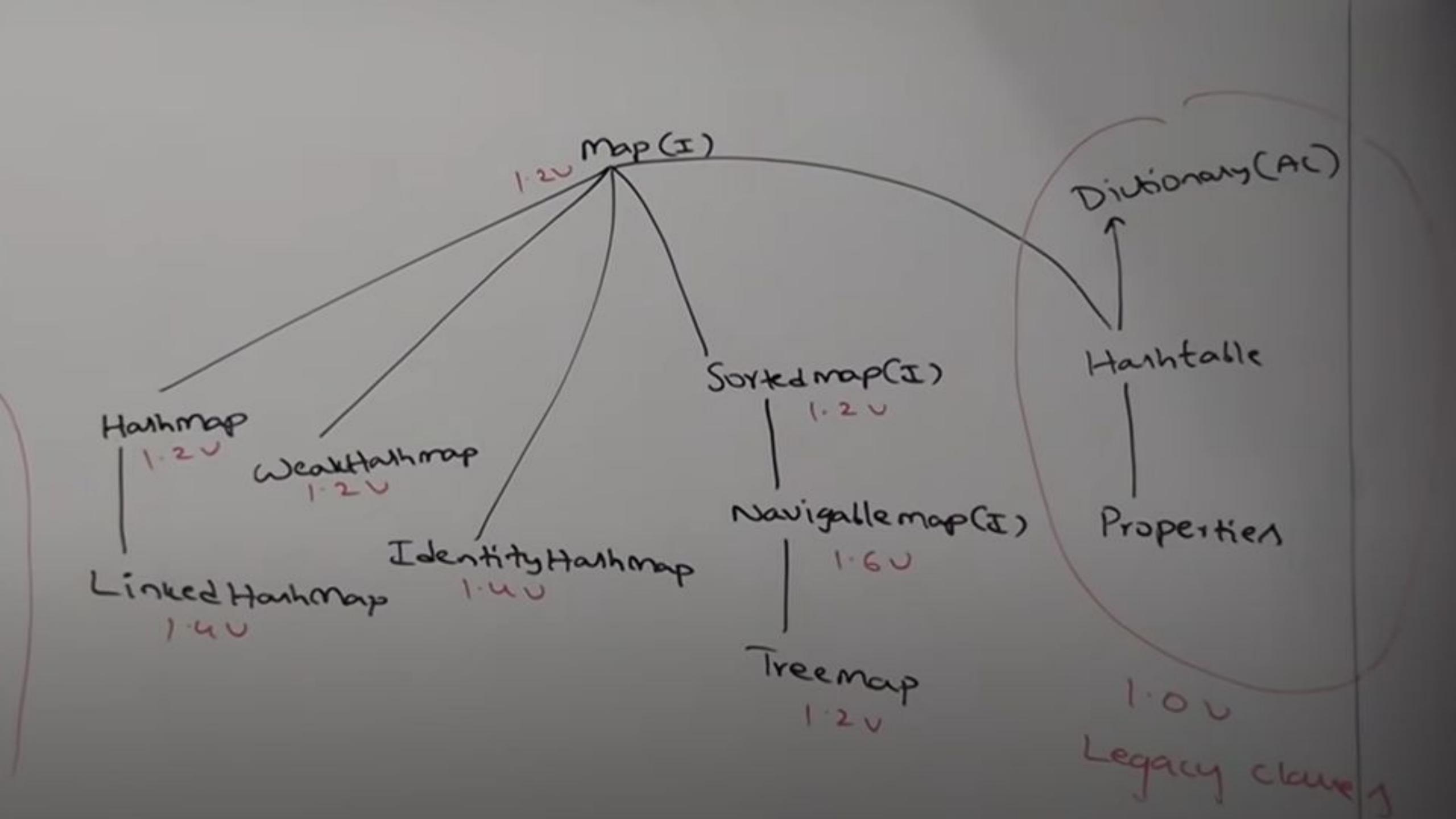
Priority Queue: Example-1

```
import java.util.*;
class PriorityQueueDemo {
      // Main Method
    public static void main(String args[])    {
        // Creating empty priority queue
        PriorityQueue<Integer> pQueue = new PriorityQueue<Integer>();

        // Adding items to the pQueue using add()
        pQueue.add(10);
        pQueue.add(20);
        pQueue.add(15);

        // Printing the top element of PriorityQueue
        System.out.println(pQueue.peek());
        // Printing the top element and removing it
        // from the PriorityQueue container
        System.out.println(pQueue.poll());
        // Printing the top element again
        System.out.println(pQueue.peek());
    }
}
```

Priority Queue: Example-2

```
import java.util.*;
import java.io.*;
public class PriorityQueueDemo {
    public static void main(String args[])    {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        for(int i=0;i<3;i++){
            pq.add(i);
            pq.add(1);        }
Iterator iterator = pq.iterator();
        while (iterator.hasNext()) {
            //System.out.print(iterator.next() + " ");
           System.out.println("Poll Method - " + pq.poll());        }
       System.out.println(pq);    }}
```

Output: Poll Method - 0
Poll Method - 1
Poll Method - 1
Poll Method - 1
Poll Method - 1
Poll Method - 2
[]


Map(I)
1.2v
Hashmap
1.2v
WeakHashmap
1.2v
IdentityHashmap
1.4v
LinkedHashMap
1.4v
SortedMap(I)
1.2v
NavigableMap(I)
1.6v
Treemap
1.2v
Dictionary(AC)
Hashtable
Properties
1.0v
Legacy class

Map

- Map is not child interface of Collection interface.
- If you want to represent a group of individual object as key value pair then we will use map.
- Key and value both are object.
- For key duplicate are not allowed but values can be duplicated.
- Implementation classes for map is HasMap,LinkedHashMap etc. as shown in next slide.
- Each key value pair is called Entry. No existence of Entry interface without map interface.
- So, map is considered as collection of entry objects.
- Collection related specific methods is not applicable in map.

Map Interface Specific methods:

1. Object put(Object key,Object value) is used to add one key value pair to the map.

If the key is already present then old value will be replaced with new value and returns old value.

If we try to put duplicate key value then old key`s value will be replaced with new value and old value will be returned as object. If we do not do then it will return null value.

Example-

m.put(101,”Sant”); returns null

m.put(102,”Ranj”); returns null

m.put(101,”Raj”); returns Sant

101-Sant Raj

102-Ranj

2. Void putAll(map m)

3.Object get(key) returns the value associated with specified key

4.Object remove(key) removes the entry associated with specified key

5. Boolean m.contains(key)

6. Boolean containsValue(value)

7. Boolean isEmpty()

8.Int m.size()

9.Void clear() all key value pair will be removed

Collection Views of Map:

10. Set keyset();

11.Collection values();

12. Set entrySet()

Entry Interface:

```
inteface map{
    interface Entry{
        Object getKeys()
        Object getValues()
        Object setValue(Object newObject)}}
```

Above three methods are called Entry specific methods and we can apply it only on Entry object.

Map Specific Methods:

```
Object getKeys()
Object getValues()
Object setValue(Object newObject)
```

HashMap:

- HashMap is implementation class of Map interface.
- Underlying data structure for HAshMap is Hash table.
- Insertion order not preserved.
- Based on hashcode of keys, all key value pairs are inserted.
- Duplicate keys not allowed
- Duplicate values are allowed
- For both keys and values heterogeneous data is allowed.
- Null is allowed for key at only once, but for values null is allowed any number of times.
- It implement both Serializable and Clonable interface but not RandomAccess.
- Best for Search operation.

Constructor:

HashMap() // default initial capacity 16 and fill ratio .75

HashMap(int initialcapacity)

HashMap(int initialcapacity,float fillratio)

HashMap(Map m)

Example-1

```
import java.util.*;
class HashMapDemo{
Public static void main(String args[]){
HashMap m=new HashMap();
m.put("Raj",700); m.put("Abu",300); m.put("Sam",800);
System.out.println(m);//{k=v,k=v}
System.out.println(m.put("Raj",1000));
Set s=m.keySet();System.out.println(s);//[k,k,k]
Collection c=m.values();System.out.println(c);//[k=v,k=v]
Set s1=m.entrySet();System.out.println(s1);
Iterator itr=s1.iterator();
while(itr.hasNext()){
Map.Entry m1=(Map.Entry)itr.next();
System.out.println(m1.getKey()+"……"+m1.getValue());
If(m1.getKey().equals("Sam")){
m1.setValue(10000);}}
System.out.println(m);}}
```

HashMap	Hashtable
1) HashMap is **non synchronized**. It is not-thread safe and can't be shared between many threads without proper synchronization code.	Hashtable is **synchronized**. It is thread-safe and can be shared with many threads.
2) HashMap **allows one null key and multiple null values**.	Hashtable **doesn't allow any null key or value**.
3) HashMap is a **new class introduced in JDK 1.2**.	Hashtable is a **legacy class**.
4) HashMap is **fast**.	Hashtable is **slow**.
5) We can make the HashMap as synchronized by calling this code Map m = Collections.synchronizedMap(hashMap);	Hashtable is internally synchronized and can't be unsynchronized.
6) HashMap is **traversed by Iterator**.	Hashtable is **traversed by Enumerator and Iterator**.
7) Iterator in HashMap is **fail-fast**.	Enumerator in Hashtable is **not fail-fast**.
8) HashMap inherits **AbstractMap** class.	Hashtable inherits **Dictionary** class.

LinkedHashMap

LinkedHashMap is a child class of HashMap.

It is exactly same as HashMap(including methods and constructors) except the below differences

Feature	HashMap	LinkedHashMap
Ordering of Elements	No order guarantees for keys or values	Maintains order based on the insertion of entries
Internal Data Structure	Hash table	Hash table + Linked list (for order)
Performance	Slightly faster for basic operations	Slightly slower due to maintenance of the linked list
Memory Usage	Lower	Higher (due to the extra pointers of the linked list)
Use-cases	When order of elements doesn't matter	When order of insertion or access matters
Iteration Order	Unpredictable	Order of insertion or optionally access order

Java Guides
Java Guides

LinkedHashMap

Example-1

```
import java.util.*;
class HashMapDemo{
Public static void main(String args[]){
LinkedHashMap m=new LinkedHashMap();
m.put("Raj",700); m.put("Abu",300); m.put("Sam",800);
System.out.println(m);//{k=v,k=v}
System.out.println(m.put("Raj",1000));
Set s=m.keySet();System.out.println(s);//[k,k,k]
Collection c=m.values();System.out.println(c);//[k=v,k=v]
Set s1=m.entrySet();System.out.println(s1);
Iterator itr=s1.iterator();
while(itr.hasNext()){
Map.Entry m1=(Map.Entry)itr.next();
System.out.println(m1.getKey()+"……"+m1.getValue());
If(m1.getKey().equals("Sam"){
m1.setValues(10000);}}
System.out.println(m);}}
```

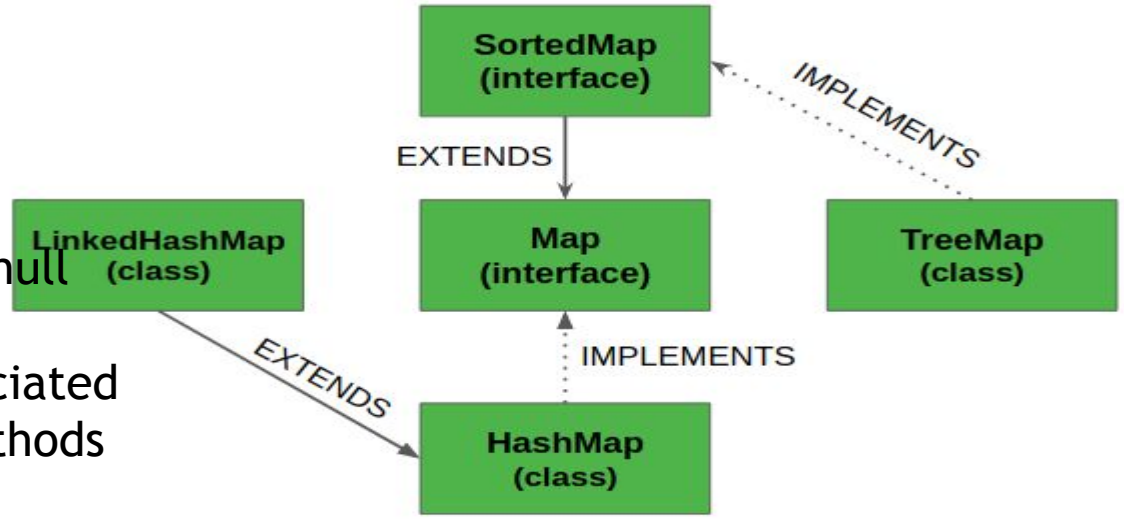
TreeMap

The TreeMap in Java is a concrete implementation of the java.util.SortedMap interface. It provides an ordered collection of key-value pairs, where the keys are ordered based on their natural order or a custom Comparator passed to the constructor.

A TreeMap is implemented using a Red-Black tree, which is a type of self-balancing binary search tree. This provides efficient performance for common operations such as adding, removing, and retrieving elements, with an average time complexity of O(log n).

Some important features of the treemap are as follows:

This class is a member of the Java Collections Framework.The class implements Map interfaces including NavigableMap, SortedMap, and extends AbstractMap class.TreeMap in Java does not allow null keys (like Map) and thus a NullPointerException is thrown. However, multiple null values can be associated with different keys.Entry pairs returned by the methods in this class and its views represent snapshots of mappings at the time they were produced. They do not support the Entry.setValue method.



MAP Hierarchy in Java

- Java TreeMap contains values based on the key. It implements the NavigableMap interface and extends AbstractMap class.
- Java TreeMap contains only unique elements.
- Java TreeMap cannot have a null key but can have multiple null values.
- Java TreeMap is non synchronized.
- Java TreeMap maintains ascending order.

Constructors in TreeMap

In order to create a TreeMap, we need to create an object of the TreeMap class. The TreeMap class consists of various constructors that allow the possible creation of the TreeMap. The following are the constructors available in this class:

TreeMap()

TreeMap(Comparator comp)

TreeMap(Map M)

TreeMap(SortedMap sm)

Constructor 1: TreeMap()

This constructor is used to build an empty treemap that will be sorted by using the natural order of its keys.

Example-1

```
import java.util.*;
import java.util.concurrent.*;
public class GFG {
        static void Example1stConstructor()      {
        TreeMap<Integer, String> tree_map= new TreeMap<Integer, String>();
        tree_map.put(10, "Geeks");          tree_map.put(15, "4");      tree_map.put(20, "Geeks");
        tree_map.put(25, "Welcomes");          tree_map.put(30, "You");
        System.out.println("TreeMap: " + tree_map);      }
        public static void main(String[] args)     {
        System.out.println("TreeMap using "+ "TreeMap() constructor:\n");
        Example1stConstructor();  }}
```

Output:

TreeMap using "+ "TreeMap() constructor:

TreeMap:{10=Geeks,15=4,20=Geeks,25-Welcome,30=You}

Example-2

```
import java.util.Map;
import java.util.TreeMap;
public class Main {
    public static void main(String[] args) {
        Map<String, Integer> treeMap = new TreeMap<>();
        // Adding elements to the tree map
        treeMap.put("A", 1);        treeMap.put("C", 3);        treeMap.put("B", 2);
        // Getting values from the tree map
        int valueA = treeMap.get("A");
        System.out.println("Value of A: " + valueA);
        // Removing elements from the tree map
        treeMap.remove("B");
        // Iterating over the elements of the tree map
        for (String key : treeMap.keySet()) {
            System.out.println("Key: " + key + ", Value: " + treeMap.get(key));    }    }}
```

Example-3

```
import java.util.*;
class TreeMap1{
 public static void main(String args[]){
   TreeMap<Integer,String> map=new TreeMap<Integer,String>();
     map.put(100,"Amit");
     map.put(102,"Ravi");
     map.put(101,"Vijay");
     map.put(103,"Rahul");
     for(Map.Entry m:map.entrySet()){
      System.out.println(m.getKey()+" "+m.getValue());
     }    }}
```

Output: 100 Amit

101 Vijay

102 Ravi

103 Rahul

Hashtable

- Underlying datastructure is Hashtable.
- Insertion order not preserved it is based on hashcode of the key.
- Duplicate keys are not allowed but values may be duplicate.
- Heterogeneous object allowed for both key and value.
- Null can not be allowed for both key and value.
- Implements Serializable and Clonable but not RandomAccess.
- Thread Safe.
- Best choice for searching.

Constructor for Hashtable

```
Hashtable h=new Hashtable();//default initial capacity is 11 and fillratio is.75

Hashtable h=new Hashtable(int initialcapacity)

Hashtable h=new Hashtable(int initialcapacity,float fillratio)

Hashtable h=new Hashtable(Map m)
```

Overriding hasCode() :

```
class Temp{

int I;

Temp(int i){this.i=i;}

public int hasCode(){return i;}

public String toString(){return i+"";}}
```

```
import java.util.*;

class HashtableDemo{

Public static void main(String[] args){

Hashtable t=new Hashtable();

h.put(new Temp(5),”A”);

h.put(new Temp(2),”B”);

h.put(new Temp(6),”C”);

h.put(new Temp(15),”D”);//15%11=4

h.put(new Temp(23),”E”);//23%11=1

h.put(new Temp(16),”F”);//16%11=5

System.out.println(h);}}
```

10	
9	
8	
7	
6	6=C
5	5=A , 16=F
4	15=D
3	
2	2=B
1	23=E
0	

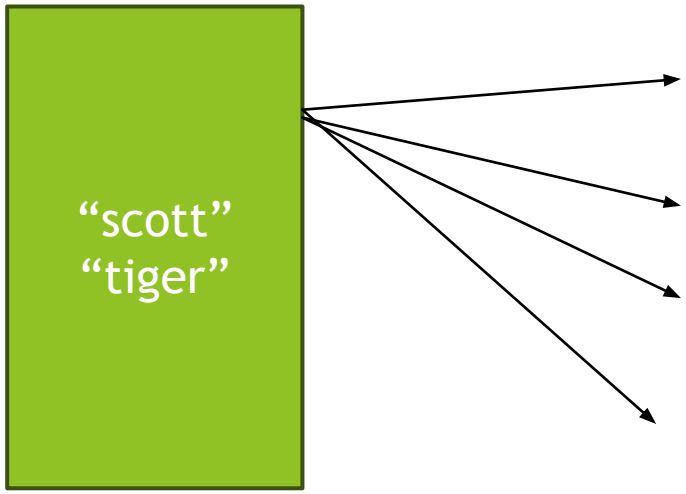
From Top to Bottom From Right to Left

```
class Temp{
int I;
Temp(int i){this.i=i;}
public int hasCode(){return i;}
public String toString(){return i+"";}}
```

Output:

{6=C,16=F,5=A,15=D,2=B,23=E}

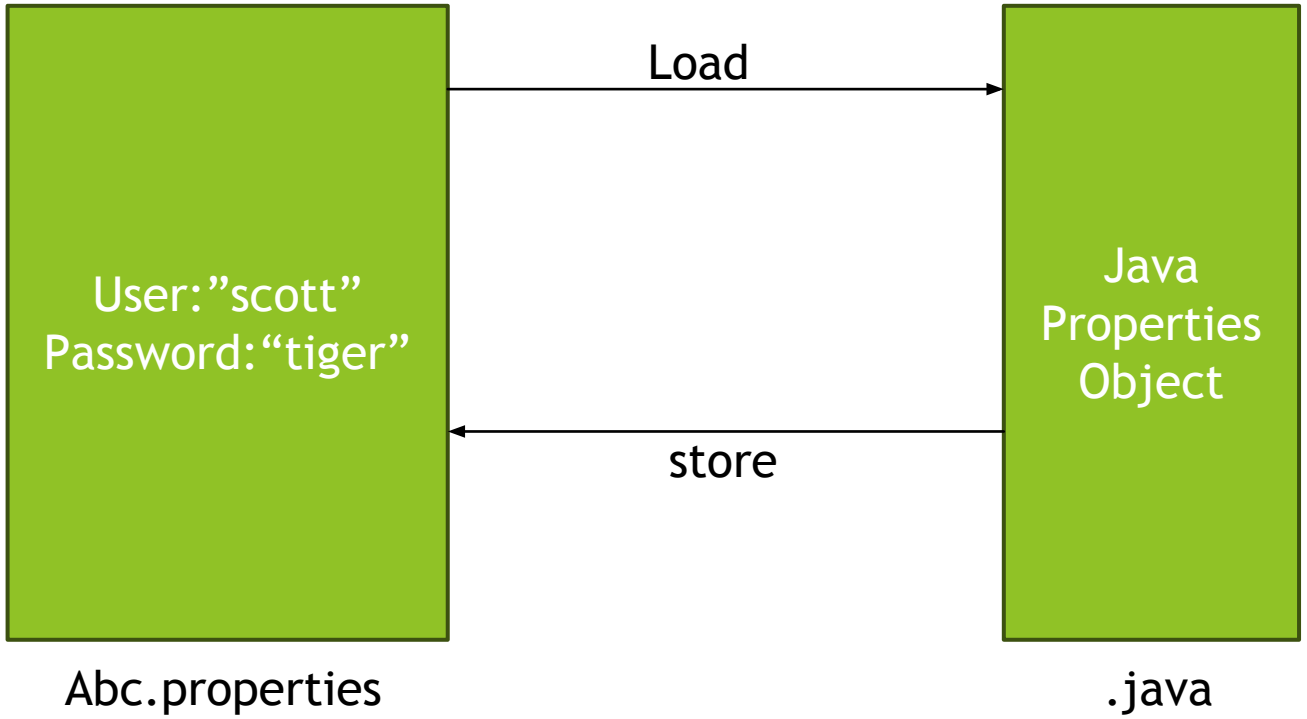
Properties Class



.java

1. Recompile
2. Rebuild(jar,war)
3. Redeploy
4. Restart the server

Anything properties that changes frequently must be put into properties file.



Such type of frequent changeable things must be configured in properties file and we have to read the data from properties file to our java program.

Construcor:

Properties p=new Properties()

The only difference between normal map(like HashMap,TreeMap etc) and Properties is that the key and value in HashMap may be of any type but in Properties class key and value both should be of String type .

Properties Specific methods:

1.String setProperty(String pname,String pvalue) // Set new property and return old valueString

2.getProperty(String pname)// to get value associated with specified property

3.Enumeration propertyNames();

4.void load(InputStream is)// load properties from properties file into java properties object.

5.void store(OutputStream os, String comment)// to store properties from java properties object into properties file

Example

```
import java.uti.*;

import java.io.*;

class PropertiesDemo{

public static void main(String[] args){

Properties p=new Properties();

FileInputStream fis=new FileInputStream(“Abc.properties”);

p.load(fis);

String s=p.getProperty(“Venki”);

System.out.println(s);

p.setProperty(“nag”,”8888”);

FileOutputStream fos=new FileOutputStream(“Abc.properties”);

p.store(fos,”Updated by Santosh Mishra”);}}
```